Contents

0.0.1 等比数列求和 . . . 2
1 组合数学 . . . 2
1.0.1 组合数预处理 . . . 2
1.0.2 斯特林数 . . . 3
1.1 群论计数 . . . 3
1.1.1 Burnside 引理 . . . 3
1.1.2 Polya 引理 . . . 4
2 数论 . . . 4
2.1 筛法 . . . 4
2.2 费马小定理 . . . 9
2.3 欧拉函数 . . . 9
2.4 欧拉定理 . . . 9
2.4.1 扩展欧拉定理 . . . 9
2.5 威尔逊定理 . . . 9
2.5.1 推论: . . . 9
2.6 裴蜀定理（贝祖定理） . . . 9
2.6.1 推广: . . . 9
2.7 扩展欧几里得算法 . . . 10
2.7.1 exgcd 求逆元 (需满足 gcd(a,mod) = 1): . . . 10
2.8 中国剩余定理 . . . 11
2.8.1 扩展中国剩余定理 . . . 11
2.9 BSGS 算法 . . . 12
3 多项式 . . . 13
3.1 拉格朗日插值 . . . 13
3.2 原根表 . . . 15
3.3 多项式全家桶 . . . 15
3.4 分治 NTT . . . 19
3.5 Chirp-Z-Transform . . . 20
4 xxx . . . 20
4.1 矩阵 . . . 21
4.2 矩阵快速幂 . . . 24
4.3 线性代数 . . . 26
4.3.1 part 1 行列式 . . . 26
4.3.2 part 2 矩阵 . . . 26
4.4 行列式求值 . . . 27
4.5 加法高斯消元 . . . 28
4.6 异或高斯消元 . . . 29
4.7 哥德巴赫猜想 . . . 31
4.8 整除分块 . . . 32
4.9 计算几何 . . . 32
4.9.1 平面几何 . . . 32
4.9.2 单峰函数与三分 . . . 48
5 概率 . . . 49
5.1 马尔可夫性 . . . 49
6 附件 . . . 49
6.1 组合数 . . . 49
6.2 因数个数 估算 . . . 50

0.0.1 等比数列求和

$$\boxed{1+q+q^2+\cdots+q^{n-1}=\frac{q^n-1}{q-1}}$$

$$\boxed{S_n=\frac{a(1-q^n)}{1-q}}$$

(q != 1)

1 组合数学

1.0.1 组合数预处理

1.0.1.1 组合数预处理 记得 init，如果模数不一样也要变, 其中 f[i] 表示 i 的阶乘，g[i] 表示 i 阶乘的逆元

```
const int mod = 998244353;
long long qpow(long long a,long long n){
    a %= mod;
    long long res = 1;
    while(n){
        if(n&1) res *= a,res %= mod;
        a *= a,a %= mod;
        n >>= 1;
    }
    return res;
}
std::vector<int> f,g;
void init(int n){
   f.resize(n+1),g.resize(n+1);
   f[0] = g[0] = 1;
   for(int i = 1;i<=n;i++) f[i] = 1LL*f[i-1]*i%mod;
   g[n] = qpow(f[n],mod-2);
   for(int i = n-1;i>=1;i--) g[i] = 1LL*(i+1)*g[i+1]%mod;
}
long long C(int a,int b){
    if(b>a || b<0) return 0;
    return 1LL*f[a]*g[b]%mod*g[a-b]%mod;
}
long long A(int a,int b){
    return 1LL*f[a]*g[a-b]%mod;
}
long long lucas(int a,int b){
    if(b>a || b<0) return 0;
    if(a == 0 || b == 0) return 1;
    return lucas(a/mod,b/mod)*C(a%mod,b%mod)%mod;
}
```

1.0.1.2 枚举固定大小的子集 gospersHack

```
    auto nxt = [&](int x)->int {
        // 生成下一个更大的同样二进制有k个1的数
        // int注意越界
        int c = x & -x;
        int r = x + c;
        return (((r ^ x) >> 2) / c) | r;
    };
    // 例如一共有10位，枚举1的个数为4的所有数字
    for(int i = (1<<4)-1;i<(1<<10);i = nxt(i)){
        cout<<i<<" ";
    }
    cout<<"\n";
```

1.0.2 斯特林数

1.0.2.1 第一类斯特林数 斯特林轮换数 实际意义: 把 n 个不同的元素，划分为 m 个非空圆排列的方案数

递推公式

$[n,m]=[n-1,m-1]+(n-1)*[n-1,m]$

从组合意义上讲即自己作为一个新的轮换 或者自己插入到已有的人的左边

边界 [0,0] = 1

```
int S[n+1][m+1];
S[0][0] = 1;
for(int i = 1;i<=n;i++){
    for(int j = 1;j<=m;j++){
        S[i][j] = (S[i-1][j-1]+1LL*(i-1)*S[i-1][j])%mod;
    }
}
```

1.0.2.2 第二类斯特林数 斯特林子集数 实际意义: 把 n 个不同的元素，划分为 m 个非空子集的方案数

递推公式

$\{n,m\}=\{n-1,m-1\}+m*\{n-1,m\}$

从组合意义上讲即自己作为一个新的子集 或者自己插入已有的子集中

常用公式

$x^n=\sum_{k=0}^{n}\{n,k\}(x,k)k!$ 常用化简 n^m

递推边界 $\{0,0\}=1$

计算公式

$\{n,m\}=\sum_{i=0}^{m}\frac{(-1)^{m-i}i^n}{i!(m-i)!}$

1.1 群论计数

主要解决一类循环对称同构计数问题

1.1.1 Burnside 引理

简单来说结论，一个置换群的等价类个数等于所有置换不动点个数的平均数

再次简化来说，就是每个置换的不动点个数的平均数就是方案数。

那么什么叫置换呢？常见的有旋转和对称同构，即我们解决的是旋转和对称情况下的方案数计数，

不动点就是指某个方案在该置换下和原方案相同，那么称作不动点.

在 $burnside$ 引理中，我们只能手动计数得到不动点个数。

$$N(G,C)=\frac{\sum_{g\in G}C(g(i))}{|G|}$$

- **圆环染色** 通过旋转 A 环可得到 B 环则视 A 和 B 为同种环

根据

$$\boxed{\text{不同染色方案数}=\frac{1}{|G|}\sum_{g\in G}|\,\text{Fix}(g)|}$$

计算 n 个元素涂 k 种颜色求有多少种本质不同的环方案有公式:

$$\boxed{A(n,k)=\frac{1}{n}\sum_{j=0}^{n-1}k^{\gcd(n,j)}}$$

直接这样计算复杂度是 $O(nlogn)$ 的，

优化: 通过按照 $\gcd(n,j)$ 的取值分组，这样就能把 n 次枚举降到枚举 n 的所有约数从而使复杂度降到 $sqrt*log$ 级别，公式如下:

$$\boxed{A(n,k)=\frac{1}{n}\sum_{d|n}\varphi\left(\frac{n}{d}\right)k^d}$$

- **项链染色** 相比圆环染色即会旋转也会对称

设有 n 颗珠子，每颗珠子有 k 种颜色。
根据 Burnside 引理：

$$Ans = \frac{1}{2n}\left(\sum_{j=0}^{n-1} k^{\gcd(n,j)} + R(n,k)\right)$$

$$\boxed{Ans = \begin{cases} \dfrac{\sum_{d|n}\varphi(n/d)k^d + nk^{(n+1)/2}}{2n}, & n \text{ 为奇数} \\ \dfrac{\sum_{d|n}\varphi(n/d)k^d + \frac{n}{2}(k^{n/2+1} + k^{n/2})}{2n}, & n \text{ 为偶数} \end{cases}}$$

1.1.2 Polya 引理

实际上是 $burnside$ 引理的一个特殊情况，我们考虑到任何一个置换都能分解为循环的乘积，那么我们将会得到一系列的置换环 或者说循环节，那么我们有结论，在 k 种颜色的情况下，一个置换 g 的不动点个数 $C(g) = k^{m(g)}$,$m(g)$ 表示置换 g 的置换环个数

2 数论

2.1 筛法

- **质数筛**:

```
std::vector<int> minp, primes;

void sieve(int n) {
    minp.assign(n + 1, 0);primes.clear();

    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            primes.push_back(i);
        }

        for (auto p : primes) {
            if (i * p > n) break;
            minp[i * p] = p;

            if (p == minp[i]) break;
        }
    }
}
```

- **区间素数筛** P1835

获取区间 [l,r] 中的所有素数，复杂度 $O(\sqrt{r}+(r-l)*loglog(r))$

```
std::vector<int> prime;

void Interval(int l, int r) {
    sieve(1e5);
    if (l == 1) l++;
    std::vector<int> a(r - l + 1);
    for (i64 p: primes) {
        i64 st = std::max(2ll, (l - 1) / p + 1) * p;
        for (i64 i = st; i <= r; i += p) {
            if (i - l >= 0) a[i - l] = 1;
        }
    }
    for (int i = 0; i <= r - l; i++) {
        if (!a[i]) prime.push_back(i + l);
    }
}
```

- 欧拉函数 (试除法)

```
int phi(int x){
    int ans = x;
    for(int i=2;i*i<=x;i++){
        if(x%i==0){
            while(x%i==0) x/=i;
            ans=ans*(i-1)/i;
        }
    }
    if(x>1) ans=ans*(x-1)/x;
    return ans;
}
```

- 欧拉函数 (筛法)

```
std::vector<i64> minp, phi, primes;

void Phi(int n) {
    minp.assign(n + 1, -1), phi.resize(n + 1), phi[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (minp[i] == -1) {
            minp[i] = i, phi[i] = i - 1;
            primes.push_back(i);
        }
        for (auto &p: primes) {
            if (i * p > n) break;
            minp[i * p] = p;
            if (minp[i] == p) {
                phi[i * p] = phi[i] * p;
                break;
            } else {
                phi[i * p] = (p - 1) * phi[i];
            }
        }
    }
}
```

- 筛法求每个数 **不同质因子的个数**

fac[i] 表示数字 i 的**不同质因子个数**

```
std::vector<int> minp, fac, primes;

void sieve(int n) {
    minp.resize(n + 1);
    fac.resize(n + 1);
    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            fac[i] = 1;
            primes.push_back(i);
        }
        for (int p: primes) {
            if (i * p > n) break;
            minp[i * p] = p;
            if (i % p == 0) {
                fac[i * p] = fac[i];
                break;
            } else {
                fac[i * p] = fac[i] + 1;
            }
        }
    }
}
```

- 筛法求有几个质因子

  fac[i] 表示数字 i 的**所有质因子个数**（包含相同质因子）

$$fac[i] = d(n) = (a_1 + 1)(a_2 + 1) \dots (a_k + 1)$$

```
std::vector<int> minp,fac,primes;
void prime_fac(int n){
    minp.resize(n+1);fac.resize(n+1);
    for(int i=2;i<=n;i++){
        if(minp[i]==0){
            minp[i]=i;
            fac[i]=1;
            primes.push_back(i);
        }
        for(int p:primes){
            if(i*p>n) break;
            minp[i*p]=p;
            fac[i*p]=fac[i]+1;
            if(i%p==0) break;
        }
    }
}
```

- 筛法求因子个数

fac[i] 表示数字 i 的**所有因子个数**

mi[i] 表示数字 i 的**最小质因子的指数**

```
std::vector<int> primes,minp,mi,fac;
void factor(int n){
    minp.resize(n+1),mi.resize(n+1),fac.resize(n+1);
    fac[1]=1;
    for(int i=2;i<=n;i++){
        if(!minp[i]){
            primes.push_back(i);
            mi[i]=1,fac[i]=2;
        }
        for(int &p:primes){
            if(p*i>n) break;
            minp[i*p]=p;
            if(i%p==0){
                mi[i*p]=mi[i]+1;
                fac[i*p]=fac[i]/mi[i*p]*(mi[i*p]+1);
        break;
            }else{
                mi[i*p]=1,fac[i*p]=fac[i]*2;
            }
        }
    }
}
```

- 筛法求因数和

$mis[i]=1+p_1^1+p_1^2+\cdots+p_1^{a_1}=\frac{p_1^{a_1+1}-1}{p_1-1}$

$facs[i]=\sigma(i)=(1+p_1+\cdots+p_1^{a_1})\times(1+p_2+\cdots+p_2^{a_2})\times\cdots$

```
std::vector<int> primes,minp,mis,facs;

void factors(int n){
    minp.resize(n+1),mis.resize(n+1),facs.resize(n+1);
    for(int i=2;i<=n;i++){
        if(minp[i]==0){
            primes.push_back(i);
            mis[i]=facs[i]=i+1;
        }
        for(int &p:primes){
            if(i*p>n) break;
            minp[i*p]=p;
            if(i%p==0){
                mis[i*p]=mis[i]*p+1;
                facs[i*p]=facs[i]/mis[i]*mis[i*p];
                break;
            }else{
                mis[i*p]=p+1;
                facs[i*p]=facs[i]*mis[i*p];
            }
        }
    }
}
```

- 筛法求莫比乌斯函数

```
std::vector<int> minp, primes, mu;

void Mobius(int n) {
    mu.resize(n + 1), minp.resize(n + 1);
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!minp[i]) {
            primes.push_back(i);
            minp[i] = i, mu[i] = -1;
        }
        for (int &p: primes) {
            if (i * p > n) break;
            minp[i * p] = p;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            } else {
                mu[i * p] = -mu[i];
            }
        }
    }
}
```

2.2 费马小定理

若 p 为质数，且 a,p 互质，则 $a^{p-1} \equiv 1(mod\ p)$

$a * a^{p-2} = a^{p-1} \equiv 1(mod\ p)$

所以快速幂求 a 模意义下的逆元就是 qpow(a,mod-2) 只适用于 p 为质数

2.3 欧拉函数

定义：1~n 中与 n 互质的数的个数，记为 $\phi(n) = \Pi_{i=1}^{s} \frac{p_i - 1}{p_i}$

```
int get_phi(int x) {
    int res = x;
    for (int i = 2; i * i <= x; i++) {
        if (x % i == 0) {
            while (x % i == 0) { x /= i; }
            res = res / i * (i - 1);
        }
    }
    if (x > 1) res = res / x * (x - 1);
    return res;
}
```

2.4 欧拉定理

若 gcd(a,m)=1 ，则 $a^{\phi(m)} \equiv 1(mod\ m)$

2.4.1 扩展欧拉定理

不需要满足 gcd(a,m) = 1

$$a^b \equiv \begin{cases} a^b & , b < \phi(m) \\ a^{b(mod\ \phi(m)) + \phi(m)} & , b \geq \phi(m) \end{cases}$$

当 gcd(a,m) !=1 时，想象一个圈，外面漏了一个尾巴的结构，起初 a 位置尾巴的位置，而圈表示周期长度是 ⊠(m) , 从 a 进入圈需要的步数不超过 ⊠(m) ,b<⊠(m) 的时候直接快速幂计算，因为终点可能还在尾巴上。否则 a 最后的位置一定落在圈内，前走 ⊠(m) 步进入圈，然后走余数步就可以了

2.5 威尔逊定理

$(p-1)! \equiv -1(mod\ p)$ 是 p 为质数的充分必要条件

2.5.1 推论:

- 若 p 是质数，则 $(p-1)! + 1 \equiv 0(modp)$
- 若 p 是大于 4 的合数，则 $(p-1)! \equiv 0(modp)$

2.6 裴蜀定理（贝祖定理）

一定存在整数 x,y 满足 ax+by=gcd(a,b)

2.6.1 推广:

- 一定存在整数 x,y 满足 ax + by = gcd(a,b) * n
- 一定存在整数 $X_1 ... X_i$ 满足 $\sum_{i=1}^{n} A_i X_i = gcd(A_1, A_2, ..., A_n)$

2.7 扩展欧几里得算法

求 ax+by=gcd(a,b) 的特解 (x_0, y_0)

```
//扩展欧几里得算法
//求出a*x+b*y=gcd(a,b)式子中，满足条件的x与y,同时求出gcd(a,b);

//x,y为经过运算最初得到的数
//先求特解(x0,y0)-->x0==x*c/gcd,y0==y*c/gcd;
//再构造通解X=x0+(b*k)/gcd(a,b),Y=y0-(a*k)/gcd(a,b)
//最小正整数解Xmin == (x0%(b/g)+(b/g))%(b/g) ,Ymin=(y0%(a/g)+(a/g))%(a/g)

i64 exgcd(i64 a, i64 b, i64 &x, i64 &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    i64 gcd = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return gcd;
}
```

2.7.1 exgcd 求逆元 (需满足 gcd(a,mod) = 1):

```
i64 getInv(i64 a, i64 mod) {
    i64 x, y;
    i64 d = exgcd(a, mod, x, y);
    if (d != 1) return -1;
    return (x % mod + mod) % mod;
}
```

2.8 中国剩余定理

求解（模数两两互质的）线性同余方程:

$$\begin{cases} x \equiv r_1 (mod\ m_1) \\ x \equiv r_2 (mod\ m_2) \\ \cdots \\ x \equiv r_n (mod\ m_n) \end{cases}$$

```
template<class T>
T exgcd(T a, T b, __int128 &x, __int128 &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    T gcd = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return gcd;
}

template<class T>
T CRT(const std::vector<T> &m, const std::vector<T> &r) {
    T M = 1, ans = 0;
    for (int i = 0; i < m.size(); i++)M *= m[i];
    for (int i = 0; i < m.size(); i++) {
        T c = M / m[i];
        __int128 x, y;
        exgcd(c, m[i], x, y);
        ans = (ans + r[i] * c * x % M) % M;
    }
    return (ans % M + M) % M;
}
```

2.8.1 扩展中国剩余定理

求解（模数两两不一定互质的）线性同余方程

```
template<class T>
T exgcd(T a, T b, __int128 &x, __int128 &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    T gcd = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return gcd;
}
template<class T>
T EXCRT(const std::vector<T> &m, const std::vector<T> &r) { //有x=r1(mod m1),x=r2(mod m2)
     //转化为x=m1*p+r1=m2*q+r2  (1)
    __int128 m1, m2, r1, r2, p, q;
    //m1*p-m2*q=r2-r1
    m1 = m[0], r1 = r[0];
    int n = m.size();
    for (int i = 1; i < n; i++) {
        m2 = m[i], r2 = r[i];
        __int128 d = exgcd(m1, m2, p, q);
        //由裴蜀定理-->> 当gcd(m1,m2)|(r2-r1)时 有解
        if ((r2 - r1) % d) { return -1; }
        //由扩欧算法得 特解p=p*(r2-r1)/gcd,q=q*(r2-r1)/gcd
        p = p * (r2 - r1) / d;//特解
        //p可能是负数，为了方便要使p变成正数,通解--》 P=p+m2/gcd*k,Q=q-m1/gcd*k
        p = (p % (m2 / d) + m2 / d) % (m2 / d);
        //x=m1*P+r1=(m1*m2)/gcd*k+m1*p1+r1,与（1）比对
        r1 = m1 * p + r1;
        //得r=m1*p+r1,m=lcm(m1,m2)
        m1 = m1 * m2 / d;
    }
    return (r1 % m1 + m1) % m1;
}
```

2.9 BSGS 算法

求解高次同余方程

给定整数 a,b,p, a,p 互质 (a,p 不互质时用 exbsgs), 求满足 $a^x \equiv b(mod\ p)$ 的最小非负整数 x

bsgs 本身复杂度是 log(p) 的，这个板子用的 map, 复杂度带个 log,tle 的话换 umap 试试

```
//a^x=b(mod p),求最小的非负整数x（a,p互质）
//根据扩展欧拉定理a^x=a^(x%phi(p)) (mod p),可知a^x模p意义下的最小循环节为phi(p)
//令x=im-j,其中m=ceil(sqrt(p)),i~(1--m),j~(0,m-1)
template<class T>
struct BSGS {
    T bsgs(T a, T b, T p) {
        a %= p, b %= p;
        if (b == 1) return 0;
        T m = ceil(sqrt((double) p)), t = b;
        std::map<T, int> doc; doc[b] = 0;
        for (int i = 1; i < m; i++) {
            t = t * a % p;
            doc[t] = i;
        }
        T mi = 1; t = 1;
        for (int i = 1; i <= m; i++) mi = mi * a % p;

        for (int i = 1; i <= m; i++) {
            t = t * mi % p;
            if (doc.count(t)) return i * m - doc[t];
        }
        return -1;
    }
    T exbsgs(T a, T b, T p) {
        a %= p, b %= p;
        if (b == 1 || p == 1) return 0;
        T k = 0, A = 1;
        while (true) { //使a,p互质
            T g = __gcd(a, p);
            if (g == 1) break;
            if (b % g != 0) return -1;
            k++, b /= g, p /= g;
            A = A * (a / g) % p;
            if (A == b) return k;
        }
        T m = ceil(sqrt((double) p)), t = b;
        std::map<T, int> doc; doc[t] = 0;
        for (int i = 1; i < m; i++) {
            t = t * a % p;
            doc[t] = i;
        }
        T mi = 1;t = A;
        for (int i = 1; i <= m; i++) mi = mi * a % p;
        for (int i = 1; i <= m; i++) {
            t = t * mi % p;
            if (doc.count(t)) return i * m - doc[t] + k;
        }
        return -1;
    }
};
```

3 多项式

3.1 拉格朗日插值

$O(n^2)$ 还原多项式 (解多元方程组用高斯消元)

```
/**
 * 拉格朗日插值还原多项式系数
 * @param pts 给定的 n 个点对 (x_i, y_i)
 * @param mod 模数（须为质数）
 * @return 长度为 n 的向量 P，P[k] 代表 x^k 的系数 a_k
 * 复杂度：时间 O(n^2)，空间 O(n)
 */
vector<int> lagrange(const vector<pair<int, int>>& pts) {
    int n = pts.size();

    // 1. 计算总乘积多项式 M(x) = (x - x_0)(x - x_1)...(x - x_{n-1})
    // M 的度数为 n，系数数组大小为 n + 1
    vector<int> M(n + 1, 0);
    M[0] = 1;
    for (int i = 0; i < n; ++i) {
        int xi = (pts[i].first % mod + mod) % mod;
        for (int j = i + 1; j >= 1; --j) {
            M[j] = (M[j - 1] - xi * M[j] % mod + mod) % mod;
        }
        M[0] = (-xi * M[0] % mod + mod) % mod;
    }

    vector<int> P(n, 0); // 最终多项式系数，P[k] 对应 x^k 的系数

    // 2. 对每个点计算基多项式与综合除法
    for (int i = 0; i < n; ++i) {
        int xi = (pts[i].first % mod + mod) % mod;
        int yi = (pts[i].second % mod + mod) % mod;

        // 计算分母 D_i = \prod_{j != i} (x_i - x_j)
        int D_i = 1;
        for (int j = 0; j < n; ++j) {
            if (i == j) continue;
            int xj = (pts[j].first % mod + mod) % mod;
            int diff = (xi - xj + mod) % mod;
            D_i = (D_i * diff) % mod;
        }

        // 标量系数 C_i = y_i / D_i
        int Ci = (yi * inv(D_i)) % mod;

        // 综合除法求 M_i(x) = M(x) / (x - x_i)
        // M_i(x) 的度数为 n-1，从高次项向低次项递推
        vector<int> Mi(n, 0);
        Mi[n - 1] = M[n];
        for (int k = n - 2; k >= 0; --k) {
            Mi[k] = (M[k + 1] + xi * Mi[k + 1]) % mod;
        }

        // 累加 C_i * M_i(x) 到总多项式 P(x) 中
        for (int k = 0; k < n; ++k) {
            P[k] = (P[k] + Ci * Mi[k]) % mod;
        }
    }

    return P;
}
```

$O(n^2)$ 算单点插值，直接用插值公式

$$P(k) = \sum_{i=0}^{n-1} y_i \prod_{j \neq i} \frac{k - x_j}{x_i - x_j} \pmod{\text{mod}}$$

```
/**
 * O(N^2) 拉格朗日插值计算单点 P(k)
 * @param points 给定的 n 个点对 (x_i, y_i)
 * @param k 要求值的点
 * @return P(k) mod mod
 */
int lagrange(const vector<pair<int, int>>& points, int k) {
    int n = points.size();
    k = (k % mod + mod) % mod;
```

```
    // 特判：若 k 恰好等于某个已知点的 x_i，直接返回对应的 y_i，防止分母为 0
    for (int i = 0; i < n; ++i) {
        int xi = (points[i].first % mod + mod) % mod;
        if (xi == k) {
            return (points[i].second % mod + mod) % mod;
        }
    }

    int ans = 0;
    for (int i = 0; i < n; ++i) {
        int xi = (points[i].first % mod + mod) % mod;
        int yi = (points[i].second % mod + mod) % mod;

        int num = 1; // 分子 \prod_{j != i} (k - x_j)
        int den = 1; // 分母 \prod_{j != i} (x_i - x_j)

        for (int j = 0; j < n; ++j) {
            if (i == j) continue;
            int xj = (points[j].first % mod + mod) % mod;

            num = num * (k - xj + mod) % mod;
            den = den * (xi - xj + mod) % mod;
        }

        // term = y_i * (num / den)
        int term = yi * num % mod * inv(den) % mod;
        ans = (ans + term) % mod;
    }

    return ans;
}
```

$O(n)$ 计算快速单点插值（前提是给定点值的横坐标是连续数点）
传入的 f 数组为 1-base, 有效的值是 f[1] 到 f[k], 分别对应 1~k 的横坐标

```
// 拉格朗日插值函数：通过已知的k个点值，计算多项式在n处的值
// 参数：
//   f: 向量，存储多项式在x=1,2,...,k处的函数值（f[1]到f[k]）
//   k: 已知点的数量（多项式次数m = k-1）
//   n: 要求解的x值
// 返回值：多项式在x=n处的值f(n)
int lagrange(std::vector<int> f, int k, int n)
{
    int res = 0; // 存储最终结果

    // 预处理前缀积和后缀积
    int pre[k + 2], suf[k + 2]; // pre[i] = (n-1)(n-2)...(n-i), suf[i] = (n-i)(n-(i+1))...(n-k)
    pre[0] = 1;                 // 前缀积初始化
    suf[k + 1] = 1;             // 后缀积初始化

    // 计算前缀积：pre[i] = Π_{j=1}^i (n-j)
    for (int i = 1; i <= k; ++i)
    {
        pre[i] = 1ll * pre[i - 1] * (n - i) % p; // 递推计算前缀积
    }

    // 计算后缀积：suf[i] = Π_{j=i}^k (n-j)
    for (int i = k; i >= 1; --i)
    {
        suf[i] = 1ll * suf[i + 1] * (n - i) % p; // 递推计算后缀积
    }

    // 组合所有部分计算拉格朗日插值结果
    for (int i = 1; i <= k; ++i)
    {
        // 计算拉格朗日基函数L_i(n)的分子部分：pre[i-1] * suf[i+1]
        // 即 Π_{j i} (n-j) = (n-1)(n-2)...(n-(i-1)) * (n-(i+1))...(n-k)
        int numerator = 1ll * pre[i - 1] * suf[i + 1] % p;

        // 计算拉格朗日基函数的分母部分：fac[i-1] * fac[k-i] * (-1)^{k-i}
        // 其中fac[i]是i的阶乘，inv()是模逆元函数
        int denominator = 1ll * inv(fac[i - 1]) * inv(fac[k - i]) % p;

        // 根据(k-i)的奇偶性决定符号
        if ((k - i) & 1)
        {
            // 当(k-i)为奇数时，符号为负
            res = (res - 1ll * f[i] * numerator % p * denominator % p + p) % p;
        }
```

```
        else
        {
            // 当(k-i)为偶数时，符号为正
            res = (res + 1ll * f[i] * numerator % p * denominator % p + p) % p;
        }
    }

    return res; // 返回插值结果f(n)
}
```

3.2 原根表

```
p=r×2^k+1
prime               r   k   g
3                   1   1   2
5                   1   2   2
17                  1   4   3
97                  3   5   5
193                 3   6   5
257                 1   8   3
7681                15  9   17
12289               3   12  11
40961               5   13  3
65537               1   16  3
786433              3   18  10
5767169             11  19  3
7340033             7   20  3
23068673            11  21  3
104857601           25  22  3
167772161           5   25  3
469762049           7   26  3
1004535809          479 21  3
2013265921          15  27  31
2281701377          17  27  3
3221225473          3   30  5
75161927681         35  31  3
77309411329         9   33  7
206158430209        3   36  22
2061584302081       15  37  7
2748779069441       5   39  3
6597069766657       3   41  5
39582418599937      9   42  5
79164837199873      9   43  5
263882790666241     15  44  7
1231453023109121    35  45  3
1337006139375617    19  46  3
3799912185593857    27  47  5
4222124650659841    15  48  19
7881299347898369    7   50  6
31525197391593473   7   52  3
180143985094819841  5   55  6
1945555039024054273 27  56  5
4179340454199820289 29  57  3
```

3.3 多项式全家桶

```
using ll = long long;

template <class T, T P>
class Polynomial : public std::vector<T>
{
    static constexpr int mul(const int &a, const int &b)
    {
        int res = 1ll * a * b % P;
        if (res < 0)
            res += P;
        return res;
    }
    static constexpr ll mul(const ll &a, const ll &b)
    {
        ll res = a * b - (ll)(1.L * a * b / P) * P;
        res %= P;
        if (res < 0)
        {
            res += P;
        }
        return res;
    }
    static constexpr T add(T a, const T &b)
    {
```

```
        a += b;
        a -= (a >= P ? P : 0);
        return a;
    }
    static constexpr T sub(T a, const T &b)
    {
        a -= b;
        a += (a < 0 ? P : 0);
        return a;
    }
    static constexpr T power(T a, ll b)
    {
        T res = 1;
        while (b)
        {
            if (b & 1)
                res = mul(res, a);
            a = mul(a, a);
            b >>= 1;
        }
        return res;
    }
    static std::vector<T> w;
    static void initW(int _log)
    {
        const int r = 1 << _log;
        if (w.size() >= r)
        {
            return;
        }
        w.assign(r, 0);
        w[r >> 1] = 1;
        T s = power(3, (P - 1) >> _log); // 3是原根

        for (int i = r / 2 + 1; i < r; i++)
        {
            w[i] = mul(w[i - 1], s);
        }

        for (int i = r / 2 - 1; i > 0; i--)
        {
            w[i] = w[i * 2];
        }
    }

public:
    using std::vector<T>::vector;

    friend void dft(Polynomial &a)
    {
        const int n = a.size();
        for (int k = n >> 1; k; k >>= 1)
        {
            for (int i = 0; i < n; i += k << 1)
            {
                for (int j = 0; j < k; j++)
                {
                    T v = a[i + j + k];
                    a[i + j + k] = mul(sub(a[i + j], v), w[k + j]);
                    a[i + j] = add(a[i + j], v);
                }
            }
        }
    }
    friend void idft(Polynomial &a)
    {
        const int n = a.size();
        for (int k = 1; k < n; k <<= 1)
        {
            for (int i = 0; i < n; i += k << 1)
            {
                for (int j = 0; j < k; j++)
                {
                    T u = a[i + j];
                    T v = mul(a[i + j + k], w[j + k]);
                    a[i + j + k] = sub(u, v);
                    a[i + j] = add(u, v);
                }
            }
        }
```

```
        T val = P - (P - 1) / n;
        for (int i = 0; i < n; i++)
        {
            a[i] = mul(a[i], val);
        }
        std::reverse(a.begin() + 1, a.end());
    }

    friend Polynomial operator*(Polynomial a, Polynomial b)
    {
        if (a.size() == 0 or b.size() == 0)
        {
            return Polynomial();
        }
        int n = a.size() + b.size() - 1;
        int _log = std::__lg(2 * n - 1);
        int s = 1 << _log;
        if (((P - 1) & (s - 1)) != 0 or std::min(a.size(), b.size()) < 128)
        {
            Polynomial res(n);
            for (int i = 0; i < a.size(); i++)
            {
                for (int j = 0; j < b.size(); j++)
                {
                    res[i + j] = add(res[i + j], mul(a[i], b[j]));
                }
            }
            return res;
        }

        initW(_log);
        a.resize(s), b.resize(s);
        dft(a), dft(b);
        for (int i = 0; i < s; i++)
        {
            a[i] = mul(a[i], b[i]);
        }
        idft(a);
        return a.resize(n), a;
    }
    friend Polynomial deriv(const Polynomial &a) // 求导
    {
        int n = a.size();
        if (n <= 1)
        {
            return Polynomial();
        }
        Polynomial p(n - 1);
        for (int i = 1; i < n; i++)
        {
            p[i - 1] = mul(i, a[i]);
        }
        return p;
    }
    friend Polynomial integr(const Polynomial &a)
    {
        int n = a.size();
        Polynomial p(n + 1);

        std::vector<int> _inv(n + 1);
        _inv[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            _inv[i] = mul(_inv[P % i], (P - P / i));
        }
        for (int i = 0; i < n; ++i)
        {
            p[i + 1] = mul(a[i], _inv[i + 1]);
        }
        return p;
    }
    friend Polynomial inv(const Polynomial &a)
    {
        int n = a.size();
        if (n == 1)
        {
            return {power(a[0], P - 2)};
        }
        Polynomial half(a.begin(), a.begin() + (n + 1) / 2);
        Polynomial b = inv(half), c = a * b;
```

```
        for (auto &x : c)
        {
            x = (x == 0 ? 0 : P - x); // ?
        }
        c[0] = add(c[0], 2);
        c = c * b;
        c.resize(n);
        return c;
    }

    friend Polynomial ln(const Polynomial &a)
    {
        int n = a.size();

        Polynomial b(n, 0);
        for (int i = 1; i < n; i++)
        {
            b[i - 1] = mul(i, a[i]);
        }
        b = b * inv(a);
        b.resize(n);

        std::vector<T> _inv(n);
        _inv[1] = 1;
        for (int i = 2; i < n; i++)
        {
            _inv[i] = mul(P - P / i, _inv[P % i]);
        }
        for (int i = n - 1; i; i--)
        {
            b[i] = mul(b[i - 1], _inv[i]);
        }
        b[0] = 0;
        return b;
    }

    friend Polynomial exp(const Polynomial &a)
    {
        int n = a.size();
        if (n == 1)
        {
            return {1};
        }
        Polynomial half(a.begin(), a.begin() + (n + 1) / 2);
        Polynomial b = exp(half);
        b.resize(n);
        Polynomial c = ln(b);
        for (int i = 0; i < n; i++)
        {
            c[i] = sub(a[i], c[i]);
        }
        c[0] = add(c[0], 1);
        c = c * b;
        c.resize(n);
        return c;
    }

    friend Polynomial power(Polynomial &F, std::string s)
    {
        T k1 = 0;
        T k2 = 0;
        T k3 = 0;
        for (int i = 0; i < s.length(); ++i)
        {
            k1 = add(mul(k1, 10), s[i] - '0');
            k2 = (1ll * k2 * 10 + s[i] - '0') % (P - 1);
            if (i < 7)
            {
                k3 = k3 * 10 + s[i] - '0';
            }
        }
        int n = F.size();
        if (!F[0] and k3 >= F.size())
        {
            F.assign(n, 0);
            return F;
        }
        T pos = 0; // 处理移位
        for (int i = 0; i < n; ++i)
        {
```

```
            if (F[i])
            {
                pos = i;
                break;
            }
        }
        if (pos)
        {
            for (int i = pos; i < n; ++i)
            {
                F[i - pos] = F[i];
                F[i] = 0;
            }
        }
        T val = F[0];
        T cur = power(val, P - 2);
        for (int i = 0; i < F.size(); ++i)
        {
            F[i] = mul(F[i], cur);
        }
        F = ln(F);
        for (int i = 0; i < F.size(); ++i)
        {
            F[i] = mul(F[i], k1);
        }
        F = exp(F);
        cur = power(val, k2);
        for (int i = 0; i < F.size(); ++i)
        {
            F[i] = mul(F[i], cur);
        }
        if (pos)
        {
            pos = (1ll * pos * k1 < n ? pos * k1 : n);
            for (int i = n - 1; i >= 0; --i)
            {
                if (i + pos < n)
                {
                    F[i + pos] = F[i];
                }
                F[i] = 0;
            }
        }
        return F;
    }

    friend Polynomial power(const Polynomial &F, int b, int m) // m是目标的区间长度
    {
        Polynomial res = {1};
        Polynomial G = F;
        while (b)
        {
            if (b & 1)
            {
                res = res * G;
                if (res.size() > m)
                {
                    res.resize(m);
                }
            }
            G = G * G;
            if (G.size() > m)
            {
                G.resize(m);
            }
            b >>= 1;
        }
        return res;
    }
};
template <class T, T P>
std::vector<T> Polynomial<T, P>::w;
using Poly = Polynomial<int, 998244353>;
```

3.4 分治 NTT

给定一个序列 $g_1, g_2 ... g_{n-1}$ 求序列 $f_0, f_1 .... f_{n-1}$ 其中 $f_i = \sum_{j=1}^{i} f_{i-j} * g_j$ 我们将给定边界 f_0

$$F(x)=\sum_{i=0}^{\infty}f_ix^i \quad G(x)=\sum_{i=0}^{\infty}g_ix^i$$

我们不妨设 设

$$F(x)G(x)=\sum_{i=0}^{\infty}x^i\sum_{j+k=i}f_jg_k=F(x)-f_0x^0(\text{因为不存在}g_0)$$

那么

$$F(x)=\frac{f_0}{1-G(x)}(modx^n)$$

因此 需要注意的是把 $G(x)$ 的每一项变成 $P-G(x)$, 就是摸 P 意义下

3.5 Chirp-Z-Transform

$$P(x)=\sum_{i=0}^{n-1}a_ix^i$$

假设我们有多项式 ，我们有常数c和m，我们希望求出

$P(c^0),P(c^1).....P(c^m)$，暴力复杂度为$O(nm)$, 我们不妨设$b_j=P(c^j)$

$$b_j=\sum_{i=0}^{n-1}a_ic^{ij}$$

代入可知 , 我们知道$ij=(i+j,2)-(i,2)-(j,2)$

化简以后可以定义$C_i=a_{n-i-1}c^{-(n-i-1,2)}$，$D_i=c^{(i,2)}$

注意D数组大小为$n+m$

$$b_jx^{(j,2)}=\sum_{i=0}^{n-1}C_{n-i}D_{i+j}$$

我们有

也就是$b_j=x^{-(j,2)}(C*D)_{n-1+j}$

```
ll n, c, m;
std::cin >> n >> c >> m;
Poly a(n);
for (auto &x : a)
{
    std::cin >> x;
}
Poly s(n), t(n + m - 1);
auto calc = [&](ll x) -> ll
{
    return 1ll * x * (x - 1) / 2;
};
for (int i = 0; i < n; ++i)
{
    s[i] = 1ll * a[n - 1 - i] * inv(qmi(c, calc(n - i - 1))) % P;
}
for (int i = 0; i < n + m - 1; ++i)
{
    t[i] = qmi(c, calc(i));
}
s = s * t;
for (int i = 0; i < m; ++i)
{
    int ans = 1ll * inv(qmi(c, calc(i))) * s[n + i - 1] % P;
    std::cout << ans << " \n"[i + 1 == m];
}
```

4 xxx

4.1 矩阵

```
using i64 = long long;
template<class T, int R, int C>
struct Matrix {
    std::array<T, R * C> A;
    int n{}, m{};

    Matrix(int _r = R, int _c = C) {
        init(_r, _c);
    }

    void init(int _r, int _c) {
        n = _r, m = _c;
        A.fill(0);
    }

    int id(int i, int j) const {
        return i * m + j;
    }

    T at(int i, int j) const {
        assert(id(i, j) >= 0 && id(i, j) < n * m);
        return A[id(i, j)];
    }

    T &operator[](int i) {
        assert(i >= 0 && i < n * m);
        return A[i];
    }

    Matrix<T, R, C> &operator=(const std::array<T, R * C> &a) {
        A = a;
        return *this;
    }

    template<int N, int M>
    Matrix<T, R, M> operator*(const Matrix<T, N, M> &rhs) {
        assert(C == N);
        Matrix<T, R, M> c(n, rhs.m);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < rhs.m; j++) {
                for (int k = 0; k < m; k++) {
                    c[c.id(i, j)] = c.at(i, j) + A[id(i, k)] * rhs.at(k, j);
                }
            }
        }
        return c;
    }

    Matrix<T, R, C> operator*(const T &a) {
        Matrix<T, R, C> c(n, m);
        for (int i = 0; i < n * m; i++) {
            c[i] = a * A[i];
        }
        return c;
    }

    friend Matrix<T, R, C> operator*(const T &a, const Matrix<T, R, C> &lhs) {
        Matrix<T, R, C> c(lhs.n, lhs.m);
        for (int i = 0; i < lhs.n * lhs.m; i++) {
            c[i] = a * lhs[i];
        }
        return c;
    }

    Matrix<T, R, C> operator+(const Matrix<T, R, C> &rhs) {
        Matrix<T, R, C> c(n, m);
        for (int i = 0; i < n * m; i++) {
            c[i] = A[i] + rhs.A[i];
        }
        return c;
    }

    Matrix<T, R, C> operator-(const Matrix<T, R, C> &rhs) {
        Matrix<T, R, C> c(n, m);
        for (int i = 0; i < n * m; i++) {
            c[i] = A[i] - rhs.A[i];
        }
        return c;
    }
```

```
    friend bool operator==(const Matrix<T, R, C> &lhs, const Matrix<T, R, C> &rhs) {
        if (lhs.n != rhs.n && lhs.m != rhs.m) return false;
        for (int i = 0; i < lhs.n * lhs.m; i++) {
            if (lhs[i] != rhs[i]) return false;
        }
        return true;
    }
    friend bool operator!=(const Matrix<T, R, C> &lhs, const Matrix<T, R, C> &rhs) {
        if (lhs.n != rhs.n && lhs.m != rhs.m) return true;
        for (int i = 0; i < lhs.n * lhs.m; i++) {
            if (lhs[i] != rhs[i]) return true;
        }
        return false;
    }
    friend constexpr std::istream &operator>>(std::istream &is, Matrix<T, R, C> &rhs) {
        for (int i = 0; i < rhs.n * rhs.m; i++) {
            is >> rhs[i];
        }
        return is;
    }

    friend constexpr std::ostream &operator<<(std::ostream &os, Matrix<T, R, C> &rhs) {
        for (int i = 0; i < rhs.n; i++) {
            for (int j = 0; j < rhs.m; j++) {
                os << rhs.at(i, j) << " ";
            }
            os << " \n"[i < rhs.n - 1];
        }
        return os;
    }
};

template<class T, int R>
Matrix<T, R, R> unit(int N) {//单位矩阵
    Matrix<T, R, R> E(N, N);
    for (int i = 0; i < N; i++) {
        E[E.id(i, i)] = 1;
    }
    return E;
}


template<class T, int R, int C>
Matrix<T, C, R> transpose(const Matrix<T, R, C> &A) {//矩阵转置
    Matrix<T, C, R> B(A.m, A.n);
    for (int i = 0; i < A.n; i++) {
        for (int j = 0; j < A.m; j++) {
            B[B.id(j, i)] = A.at(i, j);
        }
    }
    return B;
}


template<class T, int R>
T det(const Matrix<T, R, R> &A) {//方阵的行列式
    int n = A.n, m = A.m;
    assert(n == m);
    Matrix<long double, R, R> B(n, m);
    for (int i = 0; i < n * m; i++) B[i] += A.at(0, i);
    for (int c = 0; c < m; c++) {
        int ok = false, r = c;
        for (r = c; r < n; r++) {
            if (B.at(r, c) != 0) {
                ok = true;
                for (int j = 0; j < n; j++) {
                    std::swap(B[B.id(c, j)], B[B.id(r, j)]);
                }
                break;
            }
        }
        if (!ok) return 0;
        r++;
        for (; r < n; r++) {
            if (B.at(r, c) == 0) continue;
            long double t = B.at(r, c) / B.at(c, c);
            for (int j = c; j < n; j++) {
                B[B.id(r, j)] -= B[B.id(c, j)] * t;
            }
```

```
        }
    }
    T ans = 1;
    for (int i = 0; i < n; i++) {
        ans *= B.at(i, i);
    }
    return ans;
}

template<class T, int R, int C>
Matrix<T, R, C> adjont(const Matrix<T, R, C> &A) {//伴随矩阵
    int n = A.n, m = A.m;
    assert(n == m);
    Matrix<T, R, C> B(n, n);
    Matrix<T, R - 1, R - 1> tmp(n - 1, n - 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int idx = 0;
            for (int x = 0; x < n; x++) {
                for (int y = 0; y < n; y++) {
                    if (x == i || y == j) continue;
                    tmp[idx++] = A.at(x, y);
                }
            }
            B[B.id(j, i)] = ((i + j) % 2 ? -1 : 1) * det(tmp);
        }
    }
    return B;
}


template<class T, int R, int C>
Matrix<T, R, C> power(Matrix<T, R, C> a, i64 b) {
    auto ans = unit<T, R>(a.n);
    assert(a.n == a.m);
    for (; b; b >>= 1, a = a * a) {
        if (b & 1) ans = ans * a;
    }
    return ans;
}
```

4.2 矩阵快速幂

```
using i64 = long long;

constexpr int N = 100;
constexpr int mod = 1e9 + 7;
int M = N;
using Matrix = std::array<i64, N * N>;

int id(int i, int j) {
    return i * M + j;
}

Matrix init(Matrix &mat, int x = 0) {
    if (x == 0) {
        mat.fill(0);
    } else {
        for (int i = 0; i < M; i++) mat[id(i, i)] = x;
    }
    return mat;
}

Matrix operator*(const Matrix &a, const Matrix &b) {
    Matrix c{};
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < M; j++) {
            for (int k = 0; k < M; k++) {
                c[id(i, j)] = (c[id(i, j)] + a[id(i, k)] * b[id(k, j)]) % mod;
            }
        }
    }
    return c;
}

Matrix operator+(const Matrix &a, const Matrix &b) {
    Matrix c{};
    for (int i = 0; i < M * M; i++) {
        c[i] = (a[i] + b[i]) % mod;
    }
    return c;
}

Matrix power(Matrix a, i64 b) {
    Matrix ans{};
    init(ans, 1);
    for (; b; b >>= 1, a = a * a) {
        if (b & 1) ans = ans * a;
    }
    return ans;
}
```

4.2.0.1 线段树 + 矩阵修改

```
#define int long long
const int inf = 1e18;
int mod = 998244353;

int id(int x,int y){
    return x*4+y;
}
using matrix = array<int,16>;
matrix mul(matrix& s,matrix& t){
    matrix res{};
    for(int i = 0;i<4;i++){
        for(int k = 0;k<4;k++){
            int tt = s[id(i,k)];
            if(!tt) continue;
            for(int j = 0;j<4;j++){
                res[id(i,j)] = res[id(i,j)]+tt*t[id(k,j)];
            }
        }
    }
    for(int i = 0;i<16;i++) res[i] %= mod;
    return res;
}

matrix A = {{
    1,0,0,0
    ,0,1,0,0
    ,0,0,1,0
```

```
    ,0,0,0,1
}};

struct info{
    int a[4];
    info operator+(info& t){
        info res = {0,0,0,0};
        for(int i = 0;i<4;i++){
            res.a[i] = a[i]+t.a[i];
            if(res.a[i]>=mod) res.a[i] -= mod;
        }
        return res;
    }
    void apply(matrix& t){
        info res = {0,0,0,0};
        for(int i = 0;i<4;i++){
            for(int j = 0;j<4;j++){
                res.a[i] = (res.a[i]+t[id(i,j)]*a[j])%mod;
            }
        }
        *this = res;
    }
};
struct SegmentTree{
#define lc u<<1
#define rc u<<1|1

    int n;
    vector<matrix> Tag;
    vector<info> Sum;
    vector<int> bl;
    SegmentTree(const vector<info> &a):n(a.size()-1),Tag(4*n),Sum(4*n),bl(4*n){
        Sum[0] = {0,0,0,0};

        function<void(int,int,int)> build = [&](int u,int l,int r){
            Sum[u] = a[l];Tag[u] = A;bl[u] = 0;
            if(l == r) return ;
            int mid = (l+r)/2;
            build(lc,l,mid);
            build(rc,mid+1,r);
            pushup(u);
        };
        build(1,1,n);
    }
    void pushup(int u){
        Sum[u] = Sum[lc] + Sum[rc];
    }
    void pushdown(int u,matrix& tag){
        Sum[u].apply(tag);
        Tag[u] = mul(tag,Tag[u]);
        bl[u] = 1;
    }
    void pushdown(int u){
        if(bl[u]){
            pushdown(lc,Tag[u]);
            pushdown(rc,Tag[u]);
            Tag[u] = A;
            bl[u] = 0;
        }
    }
    // info rangeQuery(int l,int r){
    //     return rangeQuery(1,1,n,l,r);
    // }
    info rangeQuery(int u,int l,int r,int x,int y){
        if(x<=l and y>=r){
            return Sum[u];
        }
        int mid = (l+r)/2;
        pushdown(u);
        info res = {0,0,0,0};
        if(x<=mid) {
            info tt = rangeQuery(lc,l,mid,x,y);
            res = res+tt;
        }
        if(y>mid) {
            info tt = rangeQuery(rc,mid+1,r,x,y);
            res = res+tt;
        }
        return res;
    }
```

```
    // void rangeModify(int l,int r, matrix& tag){
    //     rangeModify(1,1,n,l,r,tag);
    // }
    void rangeModify(int u,int l,int r,int x,int y, matrix& tag){
        if(x<=l and y>=r){
            pushdown(u,tag);
            return ;
        }
        int mid = (l+r)/2;
        pushdown(u);
        if(x<=mid) rangeModify(lc,l,mid,x,y,tag);
        if(y>mid) rangeModify(rc,mid+1,r,x,y,tag);
        pushup(u);
    }
};
void solve(){
    int n;cin>>n;
    vector<info> a(n+1);
    for(int i = 1;i<=n;i++){
        for(int j = 0;j<3;j++){
            cin>>a[i].a[j];
        }
        a[i].a[3] = 1;
    }
    SegmentTree tr(a);
    int m;cin>>m;

    while(m--){
        int o;cin>>o;
        int l,r;cin>>l>>r;
        matrix opt = A;
        if(o == 7){
            info res = tr.rangeQuery(1,1,n,l, r);
            for(int i = 0;i<3;i++){
                cout<<res.a[i]<<" ";
            }
            cout<<"\n";
        }else{
            if(o == 1){
                opt[id(0,1)] = 1;
            }else if(o == 2){
                opt[id(1,2)] = 1;
            }else if(o == 3){
                opt[id(2,0)] = 1;
            }else if(o == 4){
                int v;cin>>v;
                opt[id(0,3)] = v;
            }else if(o == 5){
                int v;cin>>v;
                opt[id(1,1)] = v;
            }else if(o == 6){
                int v;cin>>v;
                opt[id(2,2)] = 0;
                opt[id(2,3)] = v;
            }
            tr.rangeModify(1,1,n,l, r, opt);
        }
    }
}
```

4.3 线性代数

4.3.1 part 1 行列式

排列：由 $1,2,3...n$ 组成的一个有序数组，叫做一个 n 级排列。
定义：在一个 n 级排列中 $a_1,a_2,...,a_n$ 中，如果较大的数 a_i 排在较小的数 a_j 的前面，则称 a_i 与 a_j 构成一个逆序。排列中逆序的个数称为它的逆序数，记作 $N(a_1,a_2,...,a_n)$
定义：逆序数为奇数的排列称为奇排列，为偶数的排列称为偶排列。
定理：一个排列经过一次对换后，奇偶性改变。
定理：n 级排列共有 n! 个，其中奇排列和偶排列各占一半。

4.3.2 part 2 矩阵

定义：m * n 个数 $a_{ij}(i=1,2,...,m;j=1,2,...,n)$ 的一个 m 行 n 列的数表，称为 m*n 矩阵。
同型矩阵：矩阵 A 与 B 是同型矩阵 <=> A 与 B 的行数相等，列数也相等。

方阵：行数等于列数的矩阵。
列矩阵：只有一列的矩阵。
零矩阵：矩阵中的数都为零的矩阵。
负矩阵：将矩阵 A 的所有元素取相反数后得到的矩阵，叫做 A 的负矩阵。
上 (下) 三角形矩阵：主对角线下 (上) 方的元素全为零的方阵。
对角形矩阵：既是上三角形矩阵，又是下三角形矩阵。

$$\begin{pmatrix} a_{11} & 0 & 0 \\ 0 & a_{22} & 0 \\ 0 & 0 & a_{33} \end{pmatrix} \qquad diag(a1, a2, ..., an)$$

单位矩阵：主对角线上的元素全为 1，其他位置的元素全为零的方阵。
矩阵加法：两个同型矩阵，对应位置的元素相加。
矩阵减法：两个同型矩阵，对应位置的元素相减。
矩阵加减法的规律与正常数组加减法的规律一样。
矩阵的数乘：数 k 乘以矩阵 A , 就是用数 k 乘以矩阵 A 的每一个元素。
矩阵转置：将矩阵 A 的各行依次变为列后得到的矩阵，A 的转置矩阵记为 A^T

(1) $(A^T)^T = A$
(2)$(A+B)^T = A^T + B^T$,$(A-B)^T = A^T - B^T$
(3) $(k*A)^T = k*A^T$
(4)$(AB)^T = B^T A^T, -> (A_1 A_2 ... A_n)^T = A_n^T ... A_2^T A_1^T$
(5)$(A^k)^T = (A^T)^k$

对称矩阵：所有的 $a_{ij} = a_{ji}$ 的方阵

(1) $A^T = A$
(2) 若 A，B 为同阶的对称矩阵则 A+B，A-B 仍为对称矩阵。
(3) 若 A，B 为同阶对称矩阵，则 AB 为对称矩阵的充要条件是 AB=BA.
(4) 对任意 n*m 矩阵 A, 则 $A^T A, AA^T$ 均为对称矩阵。

反对称矩阵：主对角线全为 0, 并且左右 $a_{ij} = -a_{ji}$ 的方阵

A 为反对称矩阵则 $A^T = -A$
(2) 若 A，B 为同阶的反对称矩阵则 A+B，A-B 仍为反对称矩阵。

$$(3)\ A \qquad k \qquad A^k \begin{cases} \text{对称矩阵，k 为偶数} \\ \text{反对称矩阵，k 为偶数} \end{cases}$$

伴随矩阵：

(1) 对于任意方阵 A , 有 $AA^* = A^*A = |A|E$
(2) 若 A 为 n 阶方阵，则 $|A^*| = |A|^{n-1}$
(3) 若 A 为方阵，则 $(A^T)^* = (A^*)^T$
(4) 若 A 为 n 阶方阵，k 为常数，则 $(kA)^* = k^{n-1}A*$

逆矩阵：设 A 是 n 阶方阵，若存在 n 阶方阵 B, 使 AB=BA=E, 则称 A 的可逆矩阵，B 为 A 的逆矩阵，记作 A^{-1} .
若方阵 A 可逆，则 A 的逆矩阵是唯一的
若 A 为 n 阶方阵，若 $|A| \neq 0$, 则称 A 使非奇异矩阵，否则 A 是奇异矩阵。
定理：方阵 A 可逆的充要条件是 A 为非奇异矩阵，并且当 A 可逆时,$A^{-1} = \frac{1}{|A|}A^*$

(1) 若 A 可逆，则 A^T 也可逆，且 $(A^T)^{-1} = (A^{-1})^T$
(2) 若 A 可逆，k 为非零常数，则 kA 也可逆，且 $(kA)^{-1} = \frac{1}{k}A^{-1}$
(3) 若 A 可逆，则 A^* 也可逆，且 $(A^*)^{-1} = (A^{-1})^* = \frac{1}{|A|}A$
(5) 若 A 可逆，m 为正整数，则 A^m 也可逆，且 $(A^m)^{-1} = (A^{-1})^m$

初等行 (列) 变换

(1) 交换矩阵的两行 (列)
(2) 用数 $k \neq 0$ 乘矩阵某一行 (列) 的所有元素
(3) 把矩阵某一行所有元素的 K 倍加到另一行 (列) 对应的元素上去

标准型矩阵：元素只有 1 和 0 组成，且矩阵的左上角是一个单位矩阵，其余元素全为零。

4.4 行列式求值

复杂度 $O(n^3 logP)$

类似于高斯消元消成上三角形式，这里我们采用了辗转相减的方法，可以应对 mod 数为非质数的情况

```
int determinant(std::vector<std::vector<int>> a, int n)
{
    int res = 1, w = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int j = i + 1; j <= n; ++j)
        {
            while (a[i][i])
            {
                int div = a[j][i] / a[i][i];
                for (int k = i; k <= n; ++k)
                {
                    a[j][k] = (a[j][k] - 1ll * div * a[i][k] % p + p) % p;
                }
                swap(a[i], a[j]);
                w = -w;
            } // 对第 i 行和第 j 行做辗转相减。不断交换次序相减
            swap(a[i], a[j]);
            w = -w;
        }
    }
    for (int i = 1; i <= n; i++) // 化成上三角
    {
        res = 1ll * a[i][i] * res % p; // 全部乘起来
    }
    res = 1ll * w * res;  // 正负问题
    return (res + p) % p;
}
```

4.5 加法高斯消元

我们通过 $O(n^3)$ 的三重循环将增广矩阵消成上三角形式，再从末尾式子开始回代得到答案，其中若系数全都为 0 但解不为 0，则说明无解。否则有解，如果存在自由元，那么有无穷多解，否则有唯一解

```
const long double eps = 1e-8;
int gauss(std::vector<std::vector<long double>> &a, int n)
{
    int col, row;
    for (col = 0, row = 0; col < n; col++) // 注意下标是从0开始的 n是未知数的个数
    {
        int tmp = row;
        for (int i = row; i < n; i++)
        {
            if (fabs(a[i][col]) > fabs(a[tmp][col]))
            {
                tmp = i;
            }
        }
        if (fabs(a[tmp][col]) < eps) // 为0
        {
            continue;
        }
        // tmp是绝对值最大的位置
        // row是最上面的位置
        // 把这两行完全交换
        for (int i = col; i <= n; i++)
        {
            std::swap(a[tmp][i], a[row][i]);
        }
        for (int i = n; i >= col; i--)
        {
            a[row][i] /= a[row][col];
        }
        for (int i = row + 1; i < n; i++)
        {
            if (fabs(a[i][col]) > eps)
            {
                for (int j = n; j >= col; j--)
                {
                    a[i][j] -= a[row][j] * a[i][col];
                }
            }
        }
        row++;
    } // 这个循环的作用是消成上三角
    for (int i = n - 1; i >= 0; i--)
    {
        for (int j = i + 1; j < n; j++)
```

```
        {
            a[i][n] -= a[i][j] * a[j][n]; // 得到正确的解
        }
    }
    if (row < n) // 这里说明[row,n-1]的行已经没系数了
    {
        for (int i = row; i < n; i++)
        {
            if (fabs(a[i][n]) > eps) // 此刻还有值就是无解
            {
                return -1; // 无解
            }
        }
        return 1; // 有无穷多组解 因为存在自由元
    }
    return 0; // 有唯一解  a[i][n]为各个解(i从0到n-1)
}
```

4.6 异或高斯消元

复杂度为 $O(n^2m/w)$，做法和加法类似，使用 $bitset$ 优化。
n 个未知数，m 个方程

```
const int N = 1001;
// rank 返回秩，K 为最小的K使前K行得到唯一解，K+1~M行无效(有唯一解的前提下)
std::vector<int> Gauss(std::vector<std::bitset<N>> &equation, std::vector<int> &res, int n,int& rank,int &K)
{
    int m = equation.size();
    std::vector<int> ans(n, 0);
    int row = 0;

    // 前向消元
    for (int col = 0; col < n && row < m; ++col)
    {
        // 寻找主元
        int pivot = -1;
        for (int i = row; i < m; ++i)
        {
            if (equation[i][col])
            {
                pivot = i;
                break;
            }
        }

        if (pivot == -1)
            continue; // 自由元

        K = max(K,pivot+1);

        // 交换行
        if (pivot != row)
        {
            std::swap(equation[row], equation[pivot]);
            std::swap(res[row], res[pivot]);
        }

        // 消去其他行
        for (int i = 0; i < m; ++i)
        {
            if (i != row && equation[i][col])
            {
                equation[i] ^= equation[row];
                res[i] ^= res[row];
            }
        }

        ++row;
    }

    // 检查无解情况
    for (int i = row; i < m; ++i)
    {
        if (res[i])
        {
            return {-1}; // 无解
        }
    }
```

```
    // 回代求解
    for (int i = 0; i < n; ++i)
    {
        ans[i] = -1; // 初始化为自由元
    }

    for (int i = row - 1; i >= 0; --i)
    {
        // 找到主元列
        int col = 0;
        while (col < n && !equation[i][col])
            ++col;

        if (col >= n)
            continue; // 全零行

        ans[col] = res[i];
        // 标记非自由元
        for (int j = col + 1; j < n; ++j)
        {
            if (equation[i][j])
            {
                if (ans[j] == -1)
                {
                    // 自由元可以任意取值，这里设为0
                    ans[j] = 0;
                }
                ans[col] ^= ans[j];
            }
        }
    }
    // 处理自由元（设为0）
    for (int i = 0; i < n; ++i)
    {
        if (ans[i] == -1)
        {
            ans[i] = 0;
        }
    }
    rank = row;
    return ans;
}

void init(){}
#define MultiTest   0
void solve(){
    int n,m;cin>>n>>m;
    std::vector<std::bitset<N>> equation(m);
    vector<int> res(m);
    for(int i = 0;i<m;i++){
        string s;cin>>s;
        for(int j = 0;j<s.size();j++){
            if(s[j] == '1') equation[i][j] = 1;
        }
        cin>>res[i];
    }
    int rank = 0,K = 0;
    auto ans = Gauss(equation, res, n, rank, K);
    if(res == vector<int>{-1}) {
            // 无解
        return ;
    }
    if(rank != n){
        cout<<"多解\n";
        return ;
    }
    cout<<K<<"\n";
    for(int i = 0;i<n;i++){
        cout<<(ans[i]&1?"1":"0")<<"\n";
    }
}
```

4.7 哥德巴赫猜想

每个大于 2 的偶数都能表示成两个质数之和
每个大于 5 的奇数都能表示成三个质数之和
如果一个奇数能表示成两个质数之和一定可以把这个数拆成 2 + 质数

4.8 整除分块

我们考虑如何解决各式各样的整除分块

我们先考虑一维的情况

即 $\sum_{i=1}^{n}\lfloor\frac{m}{f(i)}\rfloor$ 其中 m 是常数 $f(i)$ 为单元函数且单调增加, 我们只需要每次求出右端点 R，然后 $L=R+1$ 即可, 问 题在于如何求出 R，我们来推式子

首先我们已经知道左端点 L，也就相当于知道了当前的值 val 也就是说 $\lfloor\frac{m}{f(L)}\rfloor=val$ 那么 R 应该满足 $R=max(i)$ 并且 $i*val\le m$，那么 $R=\lfloor\frac{m}{val}\rfloor=\lfloor\frac{m}{\lfloor\frac{m}{f(L)}\rfloor}\rfloor$

然后我们考虑二维的情况，一般形式为 $\sum_{i=1}^{min(n,m)}\lfloor\frac{n}{i}\rfloor\lfloor\frac{m}{i}\rfloor$，在这种情况下我们发现有两个限制，但我们只需要每次操作取他们的交集即可，因为有 $L=R+1$ 在。可以保证全覆盖

也就是取 $R=min(\lfloor\frac{n}{\lfloor\frac{n}{i}\rfloor}\rfloor,\lfloor\frac{m}{\lfloor\frac{m}{i}\rfloor}\rfloor)$，至于函数的形式我们参考一维即可

```
std::vector<std::pair<ll, ll>> floorBlock(ll n, ll m)
// n/i i从1到m的整除分块区间
{
    std::vector<std::pair<ll, ll>> res;
    for (ll l = 1, r; l <= std::min(n, m); l = r + 1)
    {
        r = std::min(n / (n / l), std::min(n, m));
        res.emplace_back(l, r);
    }
    return res;
}
```

上取整的整除分块

```
std::vector<std::pair<ll, ll>> ceilBlock(ll n)
// n/i向上取整的分块区间
{
    std::vector<std::pair<ll, ll>> res;
    for (ll l = 1, r; l <= n; l = r + 1)
    {
        ll val = (n + l - 1) / l;
        r = (val == 1 ? n : (n - 1) / (val - 1));
        res.emplace(l, r);
    }
    return res;
}
```

4.9 计算几何

4.9.1 平面几何

4.9.1.1 多边形几何重心坐标公式 对于逆时针排列的多边形顶点 $P_i=(x_i,y_i)$,

定义 $c_i=x_iy_{i+1}-x_{i+1}y_i$.

那么有向面积：$2S=\sum_{i=1}^{N}c_i$.

而几何重心直接是：

$\bar{x}=\frac{\sum_{i=1}^{N}(x_i+x_{i+1})c_i}{3\sum_{i=1}^{N}c_i}$	$\bar{y}=\frac{\sum_{i=1}^{N}(y_i+y_{i+1})c_i}{3\sum_{i=1}^{N}c_i}$

下标循环：$P_{N+1}=P_1$.

4.9.1.2 板子

```
using ll = long long;
using ld = long double;  // 浮点数类型
const ld eps = 1e-10;    // 浮点精度阈值
const ld pi = acosl(-1); //
using pld = std::pair<ld, ld>;
using pii = std::pair<int, int>;
using pll = std::pair<ll, ll>;

// 符号函数：返回x的符号（1/0/-1）
int sgn(ld x)
{
    if (x > eps)
        return 1;
    if (x < -eps)
        return -1;
    return 0;
}
```

```cpp
// 基础二维点/向量结构
template <class T>
struct pt
{
    T x, y;
    // 向量运算
    bool operator==(const pt &a) const { return (fabs(x - a.x) <= eps && fabs(y - a.y) <= eps); }
    bool operator<(const pt &a) const
    {
        if (fabs(x - a.x) <= eps)
            return y < a.y - eps;
        return x < a.x - eps;
    }
    bool operator>(const pt &a) const { return !(*this < a || *this == a); }
    pt operator+(const pt &a) const { return {x + a.x, y + a.y}; } // 向量加
    pt operator-(const pt &a) const { return {x - a.x, y - a.y}; } // 向量减
    pt operator-() const { return {-x, -y}; }                      // 向量取反
    pt operator*(const T k) const { return {k * x, k * y}; }
    pt operator/(const T k) const { return {x / k, y / k}; }
    // 点积（*）、叉积（%）
    T operator*(const pt &a) const { return x * a.x + y * a.y; }                                                    // 点积
    T operator%(const pt &a) const { return x * a.y - y * a.x; }                                                    // 叉积
    T len2() const { return (*this) * (*this); }                                                                    // 向量长度的平方
    T dis2(const pt &a) const { return (a - (*this)).len2(); }                                                      // 两点距离的平方
    ld len() const { return sqrtl(len2()); }                                                                        // 向量长度
    ld dis(const pt &a) const { return sqrtl(dis2(a)); }                                                            // 两点距离
    ld ang(const pt &a) const { return acosl(std::max(-1.0l, std::min(1.0l, ((*this) * a) / (len() * a.len())))); } // 向量夹角
    int toleft(const pt &a) const
    {
        const auto t = (*this) % a;
        return (t > eps) - (t < -eps);
    } // to-left 测试
    // 检查a向量在this的哪一测 >0逆时针 <0顺时针 =0同向

    // 旋转：90度逆时针
    pt rotate90() const { return {-y, x}; } // 逆时针旋转90°
    pt rotate(ld theta) const
    {
        ld c = cosl(theta), s = sinl(theta);
        return {
            (x * c - y * s),
            (x * s + y * c)};
    }
    pt rotate(const ld cosr, const ld sinr) const { return {x * cosr - y * sinr, x * sinr + y * cosr}; } // 逆时针旋转（给定角度的正弦与余弦）
};
// 极角排序
template <class T>
struct argcmp
{
    bool operator()(const pt<T> &a, const pt<T> &b) const
    {
        const auto quad = [](const pt<T> &a)
        {
            if (a.y < -eps)
                return 1;
            if (a.y > eps)
                return 4;
            if (a.x < -eps)
                return 5;
            if (a.x > eps)
                return 3;
            return 2;
        };
        const int qa = quad(a), qb = quad(b);
        if (qa != qb)
            return qa < qb;
        const auto t = a % b;
        if (fabs(t) <= eps)
            return a * a < b * b - eps; // 不同长度的向量需要分开
        return t > eps;
    }
};
template <class T>
struct line // 直线
{
```

```
    pt<T> p, v; // p 为直线上一点，v 为方向向量

    bool operator==(const line &a) const { return v.toleft(a.v) == 0 && v.toleft(p - a.p) == 0; }
    int toleft(const pt<T> &a) const { return v.toleft(a - p); } // to-left 测试 点a和该直线的位置关系
    bool operator<(const line &a) const                          // 半平面交算法定义的排序
    {
        if (fabs(v % a.v) <= eps && v * a.v >= -eps)
            return toleft(a.p) == -1;
        return argcmp<T>()(v, a.v);
    }
    // 涉及浮点数
    ld dis(const pt<T> &a) const { return fabs(v % (a - p)) / v.len(); } // 点到直线距离
    // ------------------------------
    // 求两直线交点
    // 返回值：是否有唯一交点
    //   false: 平行或重合
    //   true : 有交点
    // ------------------------------
    bool isCross(const line &a) const // 是否有交
    {
        ld d = v % a.v;
        return sgn(d) ? 1 : 0;
    }
    pt<T> inter(const line &a) const // 求交
    {
        ld d = v % a.v;
        ld t = (a.v % (p - a.p)) / d;
        return p + v * t;
    }
    // ------------------------------
    // 点到直线的垂足
    // 参数：q 点
    // 返回：q 在直线上的投影点（垂足）
    // ------------------------------
    pt<T> foot(const pt<T> &q) const // 垂足
    {
        // 向量 p->q
        pt<T> u = q - p;
        // 投影系数：t = (u • v) / (v • v)
        ld t = 1.L * (u * v) / (v * v);
        return p + v * t;
    }
    // 点a关于直线bc的对称点
    pt<T> symmetry(pt<T> a) const
    {
        // 先求垂足
        pt<T> f = foot(a);
        // 对称点：f + (f - a) = 2f - a
        return f * 2 - a;
    }
};
template <class T>
struct Segment
{
    pt<T> a, b;
    // 判断点a是否在线段bc上
    //-1在端点上 0不在 1在线段中间
    int onSeg(pt<T> p) const
    {
        if (p == a || p == b)
            return -1;
        return (p - a).toleft(p - b) == 0 && (p - a) * (p - b) < -eps;
    }
    // 判断线段直线是否相交
    // -1 直线经过线段端点 | 0 线段和直线不相交 | 1 线段和直线严格相交
    int isCross1(const line<T> &c) const
    {
        if (c.toleft(a) == 0 || c.toleft(b) == 0)
        {
            return -1;
        }
        return c.toleft(a) != c.toleft(b);
    }
    pt<T> inter1(const line<T> &c) const // 需要保证有交
    {
        if (c.toleft(a) == 0)
        {
            return a;
        }
```

```
        if (c.toleft(b) == 0)
        {
            return b;
        }
        ld cur = 1.L * (c.v % (c.p - a)) / (c.v % (b - a));
        return a + cur * (b - a);
    }
    // 判断两线段是否相交
    // -1 在某一线段端点处相交 | 0 两线段不相交 | 1 两线段严格相交
    int isCross2(const Segment &s) const
    {
        if (onSeg(s.a) || onSeg(s.b) || s.onSeg(a) || s.onSeg(b))
        {
            return -1;
        }
        line<T> l{a, b - a}, ls{s.a, s.b - s.a};
        return l.toleft(s.a) * l.toleft(s.b) == -1 && ls.toleft(a) * ls.toleft(b) == -1;
    }
    pt<T> inter2(const Segment &s) const // 需要保证有交
    {
        if (onSeg(s.a))
        {
            return s.a;
        }
        if (onSeg(s.b))
        {
            return s.b;
        }
        if (s.onSeg(a))
        {
            return a;
        }
        if (s.onSeg(b))
        {
            return b;
        }
        pt<T> u = b - a;     // 向量 AB
        pt<T> v = s.b - s.a; // 向量 CD
        pt<T> w = s.a - a;   // 向量 AC
        // t = (AC × v) / (u × v)
        ld t = 1.L * (w % v) / (u % v);
        return a + t * u;
    }
    // 点到线段距离
    ld dis(const pt<T> &p) const
    {
        if ((p - a) * (b - a) < -eps || (p - b) * (a - b) < -eps)
        {
            return std::min(p.dis(a), p.dis(b));
        }
        const line<T> l{a, b - a};
        return l.dis(p);
    }
    // 两线段间最近两点的距离
    ld dis(const Segment &s) const
    {
        if (isCross2(s))
        {
            return 0;
        }
        return std::min({dis(s.a), dis(s.b), s.dis(a), s.dis(b)});
    }
};
template <class T>
struct Polygon
{
    std::vector<pt<T>> p;
    int nxt(int t) const
    {
        return t == p.size() - 1 ? 0 : t + 1;
    }
    int pre(int t) const
    {
        return t == 0 ? p.size() - 1 : t - 1;
    }
    std::pair<bool, int> winding(const pt<T> &a) const
    {
        int cnt = 0;
        for (int i = 0; i < p.size(); i++)
        {
```

```
            const pt<T> u = p[i], v = p[nxt(i)];
            if (fabs((a - u) % (a - v)) <= eps && (a - u) * (a - v) <= eps)
                return {true, 0};
            if (fabs(u.y - v.y) <= eps)
                continue;
            const line<T> uv = {u, v - u};
            if (u.y < v.y - eps && uv.toleft(a) <= 0)
                continue;
            if (u.y > v.y + eps && uv.toleft(a) >= 0)
                continue;
            if (u.y < a.y - eps && v.y >= a.y - eps)
                cnt++;
            if (u.y >= a.y - eps && v.y < a.y - eps)
                cnt--;
        }
        return {false, cnt};
    }

    // 0表示在上面  -1表示在外面 1表示在里面
    int operator()(const pt<T> &a) const // 点是否在多边形内 回转数法
    {
        auto [onEdge, cnt] = winding(a);
        if (onEdge)
            return 0;
        return (cnt != 0 ? 1 : -1);
    }
    // 多边形面积的两倍
    // 可用于判断点的存储顺序是顺时针或逆时针
    // 如果是负数，说明当前是顺时针
    T area() const
    {
        T sum = 0;
        for (int i = 0; i < p.size(); i++)
            sum += p[i] % p[nxt(i)];
        return sum;
    }
    // 多边形的周长
    ld circ() const
    {
        ld sum = 0;
        for (int i = 0; i < p.size(); i++)
            sum += p[i].dis(p[nxt(i)]);
        return sum;
    }
};

template <class T>
struct Convex
{
    std::vector<pt<T>> p;
    int size() const
    {
        return p.size();
    }
    int nxt(int t) const
    {
        return t == p.size() - 1 ? 0 : t + 1;
    }
    int pre(int t) const
    {
        return t == 0 ? p.size() - 1 : t - 1;
    }
    // 闵可夫斯基和
    // 复杂度为 O(N + M),必须保证两个凸包都按逆时针顺序排列
    Convex operator+(const Convex &c) const
    {
        const auto &p = this->p;
        std::vector<Segment<T>> e1(p.size()), e2(c.p.size()), edge(p.size() + c.p.size());
        std::vector<pt<T>> res;
        res.reserve(p.size() + c.p.size());
        const auto cmp = [](const Segment<T> &u, const Segment<T> &v)
        { return argcmp<T>()(u.b - u.a, v.b - v.a); };
        for (int i = 0; i < p.size(); i++)
            e1[i] = {p[i], p[this->nxt(i)]};
        for (int i = 0; i < c.p.size(); i++)
            e2[i] = {c.p[i], c.p[c.nxt(i)]};
        std::rotate(e1.begin(), std::min_element(e1.begin(), e1.end(), cmp), e1.end());
        std::rotate(e2.begin(), std::min_element(e2.begin(), e2.end(), cmp), e2.end());
        std::merge(e1.begin(), e1.end(), e2.begin(), e2.end(), edge.begin(), cmp);
        const auto check = [](const std::vector<pt<T>> &res, const pt<T> &u)
```

```
        {
            const auto b1 = res.back(), b2 = *std::prev(res.end(), 2);
            return (b1 - b2).toleft(u - b1) == 0 && (b1 - b2) * (u - b1) >= -eps;
        };
        auto u = e1[0].a + e2[0].a;
        for (const auto &v : edge)
        {
            while (res.size() > 1 && check(res, u))
                res.pop_back();
            res.push_back(u);
            u = u + v.b - v.a;
        }
        if (res.size() > 1 && check(res, res[0]))
            res.pop_back();
        return {res};
    }
    // 旋转卡壳
    // func 为更新答案的函数，可以根据题目调整位置
    // 利用双指针，对每个边找到离这个边最远的点，复杂度O(n)
    template <typename F>
    void rotcaliper(const F &func) const
    {
        const auto &p = this->p;
        const auto area = [](const pt<T> &u, const pt<T> &v, const pt<T> &w)
        { return (w - u) % (w - v); };
        for (int i = 0, j = 1; i < p.size(); i++)
        {
            const auto nxti = this->nxt(i);
            func(p[i], p[nxti], p[j]);
            while (area(p[this->nxt(j)], p[i], p[nxti]) >= area(p[j], p[i], p[nxti]))
            {
                j = this->nxt(j);
                func(p[i], p[nxti], p[j]);
            }
        }
    }

    // 凸多边形的直径的平方
    T diameter2() const
    {
        const auto &p = this->p;
        if (p.size() == 1)
            return 0;
        if (p.size() == 2)
            return p[0].dis2(p[1]);
        T ans = 0;
        auto func = [&](const pt<T> &u, const pt<T> &v, const pt<T> &w)
        {
            ans = std::max({ans, w.dis2(u), w.dis2(v)});
        };
        rotcaliper(func);
        return ans;
    }
    // 判断点是否在凸多边形内
    // 复杂度 O(logn)
    // -1 点在多边形边上 / 0 点在多边形外 / 1 点在多边形内
    int operator()(const pt<T> &a) const
    {
        const auto &p = this->p;
        if (p.size() == 1)
            return a == p[0] ? -1 : 0;
        if (p.size() == 2)
            return Segment<T>{p[0], p[1]}.onSeg(a) ? -1 : 0;
        if (a == p[0])
            return -1;
        if ((p[1] - p[0]).toleft(a - p[0]) == -1 || (p.back() - p[0]).toleft(a - p[0]) == 1)
            return 0;
        const auto cmp = [&](const pt<T> &u, const pt<T> &v)
        { return (u - p[0]).toleft(v - p[0]) == 1; };
        const int i = std::lower_bound(p.begin() + 1, p.end(), a, cmp) - p.begin();
        if (i == 1)
            return Segment<T>{p[0], p[i]}.onSeg(a) ? -1 : 0;
        if (i == p.size() - 1 && Segment<T>{p[0], p[i]}.onSeg(a))
            return -1;
        if (Segment<T>{p[i - 1], p[i]}.onSeg(a))
            return -1;
        return (p[i] - p[i - 1]).toleft(a - p[i - 1]) > 0;
    }

    // 凸多边形关于某一方向的极点
```

```
    // 复杂度 O(logn)
    // 参考资料：https://codeforces.com/blog/entry/48868
    // 给定一个方向，用与该方向垂直的支撑线去顶凸多边形，最后接触的点就是这个方向的极点
    // 返回的是凸多边形中某个极点(可能一个边上都是极点，返回一个端点)的下标
    template <typename F>
    int extreme(const F &dir) const
    {
        const auto &p = this->p;
        const auto check = [&](const int i)
        { return dir(p[i]).toleft(p[this->nxt(i)] - p[i]) >= 0; };
        const auto dir0 = dir(p[0]);
        const auto check0 = check(0);
        if (!check0 && check(p.size() - 1))
            return 0;
        const auto cmp = [&](const pt<T> &v)
        {
            const int vi = &v - p.data();
            if (vi == 0)
                return 1;
            const auto checkv = check(vi);
            const auto t = dir0.toleft(v - p[0]);
            if (vi == 1 && checkv == check0 && t == 0)
                return 1;
            return checkv % (checkv == check0 && t <= 0);
        };
        return std::partition_point(p.begin(), p.end(), cmp) - p.begin();
    }

    // 过凸多边形外一点求凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    // 必须保证点在多边形外
    std::pair<int, int> tangent(const pt<T> &a) const
    {
        const int i = extreme([&](const pt<T> &u)
                              { return u - a; });
        const int j = extreme([&](const pt<T> &u)
                              { return a - u; });
        return {i, j};
    }

    // 求平行于给定直线的凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    std::pair<int, int> tangent(const line<T> &a) const
    {
        const int i = extreme([&](...)
                              { return a.v; });
        const int j = extreme([&](...)
                              { return -a.v; });
        return {i, j};
    }
};
// 点集的凸包
// Andrew 算法，复杂度 O(nlogn),返回的凸包顶点完全是按逆时针
template <class T>
std::vector<pt<T>> convexhull(std::vector<pt<T>> p)
{
    std::vector<pt<T>> st;
    if (p.empty())
    {
        return std::vector<pt<T>>{};
    }
    std::sort(p.begin(), p.end());
    const auto check = [](const std::vector<pt<T>> &st, const pt<T> &u)
    {
        const auto b1 = st.back(), b2 = *prev(st.end(), 2);
        return (b1 - b2).toleft(u - b1) <= 0;
    };
    for (const auto &u : p)
    {
        while (st.size() > 1 && check(st, u))
            st.pop_back();
        st.push_back(u);
    }
    int k = st.size();
    p.pop_back();
    std::reverse(p.begin(), p.end());
    for (const auto &u : p)
    {
        while (st.size() > k && check(st, u))
            st.pop_back();
```

```
        st.push_back(u);
    }
    st.pop_back();
    return st;
}
template <class T>
struct Circle
{
    pt<T> c;
    T r;
    bool operator==(const Circle &a) const { return c == a.c && fabs(r - a.r) <= eps; }
    ld circ() const { return 2 * pi * r; } // 周长
    ld area() const { return pi * r * r; } // 面积

    // 点与圆的关系
    // -1 圆上 / 0 圆外 / 1 圆内
    int is_in(const pt<T> &p) const
    {
        const ld d = p.dis(c);
        return fabs(d - r) <= eps ? -1 : d < r - eps;
    }
    // 直线与圆关系
    // 0 相离 / 1 相切 / 2 相交
    int relation(const line<T> &l) const
    {
        const ld d = l.dis(c);
        if (d > r + eps)
            return 0;
        if (fabs(d - r) <= eps)
            return 1;
        return 2;
    }
    // 圆与圆关系
    // -1 相同 / 0 相离 / 1 外切 / 2 相交 / 3 内切 / 4 内含
    int relation(const Circle<T> &a) const
    {
        if (*this == a)
            return -1;
        const ld d = c.dis(a.c);
        if (d > r + a.r + eps)
            return 0;
        if (fabs(d - r - a.r) <= eps)
            return 1;
        if (fabs(d - fabs(r - a.r)) <= eps)
            return 3;
        if (d < fabs(r - a.r) - eps)
            return 4;
        return 2;
    }

    // 直线与圆的交点
    std::vector<pt<T>> inter(const line<T> &l) const
    {
        const ld d = l.dis(c);
        const pt<T> p = l.foot(c);
        const int t = relation(l);
        if (t == 0)
            return std::vector<pt<T>>();
        if (t == 1)
            return std::vector<pt<T>>{p};
        const ld k = sqrtl(r * r - d * d);
        return std::vector<pt<T>>{p - (l.v / l.v.len()) * k, p + (l.v / l.v.len()) * k};
    }
    // 圆与圆交点
    std::vector<pt<T>> inter(const Circle &a) const
    {
        const ld d = c.dis(a.c);
        const int t = relation(a);
        if (t == -1 || t == 0 || t == 4)
            return std::vector<pt<T>>();
        pt<T> e = a.c - c;
        e = e / e.len() * r;
        if (t == 1 || t == 3)
        {
            if (r * r + d * d - a.r * a.r >= -eps)
                return std::vector<pt<T>>{c + e};
            return std::vector<pt<T>>{c - e};
        }
        const ld costh = (r * r + d * d - a.r * a.r) / (2 * r * d), sinth = sqrtl(1 - std::min(1.l, costh * costh));
        return std::vector<pt<T>>{c + e.rotate(costh, -sinth), c + e.rotate(costh, sinth)};
```

```
    }
    // 圆与圆交面积
    ld inter_area(const Circle &a) const
    {
        const ld d = c.dis(a.c);
        const int t = relation(a);
        if (t == -1)
            return area();
        if (t < 2)
            return 0;
        if (t > 2)
            return std::min(area(), a.area());
        const ld costh1 = (r * r + d * d - a.r * a.r) / (2 * r * d), costh2 = (a.r * a.r + d * d - r * r) / (2 * a.r * d);
        const ld sinth1 = sqrtl(1 - std::min(costh1 * costh1, 1.l)), sinth2 = sqrtl(1 - std::min(costh2 * costh2, 1.l));
        const ld th1 = acosl(costh1), th2 = acosl(costh2);
        return r * r * (th1 - costh1 * sinth1) + a.r * a.r * (th2 - costh2 * sinth2);
    }
    // 过圆外一点圆的切线
    std::vector<line<T>> tangent(const pt<T> &a) const
    {
        const int t = is_in(a);
        if (t == 1)
            return std::vector<line<T>>();
        if (t == -1)
        {
            const pt<T> v = {-(a - c).y, (a - c).x};
            return std::vector<line<T>>{{a, v}};
        }
        pt<T> e = a - c;
        e = e / e.len() * r;
        const long double costh = r / c.dis(a), sinth = sqrtl(1 - std::min(costh * costh, 1.l));
        const pt<T> t1 = c + e.rotate(costh, -sinth), t2 = c + e.rotate(costh, sinth);
        return std::vector<line<T>>{{a, t1 - a}, {a, t2 - a}};
    }
    // 两圆的公切线
    std::vector<line<T>> tangent(const Circle &a) const
    {
        const int t = relation(a);
        std::vector<line<T>> lines;
        if (t == -1 || t == 4)
            return lines;
        if (t == 1 || t == 3)
        {
            const pt<T> p = inter(a)[0], v = {-(a.c - c).y, (a.c - c).x};
            lines.push_back({p, v});
        }
        const ld d = c.dis(a.c);
        const pt<T> e = (a.c - c) / (a.c - c).len();
        if (t <= 2)
        {
            const ld costh = (r - a.r) / d, sinth = sqrtl(1 - std::min(costh * costh, 1.l));
            const pt<T> d1 = e.rotate(costh, -sinth), d2 = e.rotate(costh, sinth);
            const pt<T> u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c + d1 * a.r, v2 = a.c + d2 * a.r;
            lines.push_back({u1, v1 - u1});
            lines.push_back({u2, v2 - u2});
        }
        if (t == 0)
        {
            const ld costh = (r + a.r) / d, sinth = sqrtl(1 - std::min(costh * costh, 1.l));
            const pt<T> d1 = e.rotate(costh, -sinth), d2 = e.rotate(costh, sinth);
            const pt<T> u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c - d1 * a.r, v2 = a.c - d2 * a.r;
            lines.push_back({u1, v1 - u1});
            lines.push_back({u2, v2 - u2});
        }
        return lines;
    }
    // | 原图形 | 与（原点）的关系 | 反演结果 |
    // | --- | ----------- | ---- |
    // | 直线 | 经过（原点） | 直线 |
    // | 直线 | 不经过（原点） | 圆 |
    // | 圆 | 经过（原点） | 直线 |
    // | 圆 | 不经过（原点） | 圆 |
    // | 圆 | 圆心为（原点） | 同心圆 |
    // 直线的圆反演
    std::tuple<int, Circle, line<T>> inverse(const line<T> &l) const
    {
        const Circle null_c = {{0.0, 0.0}, 0.0};
        const line<T> null_l = {{0.0, 0.0}, {0.0, 0.0}};
        if (l.toleft(c) == 0)
            return {2, null_c, l};
```

```
        const pt<T> v = l.toleft(c) == 1 ? pt<T>{l.v.y, -l.v.x} : pt<T>{-l.v.y, l.v.x};
        const ld d = r * r / l.dis(c);
        const pt<T> p = c + v / v.len() * d;
        return {1, {(c + p) / 2, d / 2}, null_l};
    }
    // 圆的圆反演
    std::tuple<int, Circle, line<T>> inverse(const Circle &a) const
    {
        const Circle null_c = {{0.0, 0.0}, 0.0};
        const line<T> null_l = {{0.0, 0.0}, {0.0, 0.0}};
        const pt<T> v = a.c - c;
        if (a.is_in(c) == -1)
        {
            const ld d = r * r / (a.r + a.r);
            const pt<T> p = c + v / v.len() * d;
            return {2, null_c, {p, {-v.y, v.x}}};
        }
        if (c == a.c)
            return {1, {c, r * r / a.r}, null_l};
        const ld d1 = r * r / (c.dis(a.c) - a.r), d2 = r * r / (c.dis(a.c) + a.r);
        const pt<T> p = c + v / v.len() * d1, q = c + v / v.len() * d2;
        return {1, {(p + q) / 2, p.dis(q) / 2}, null_l};
    }
};

// 圆与多边形面积交
template <class T>
ld area(const Circle<T> &circ, const Polygon<T> &poly)
{
    const auto cal = [](const Circle<T> &circ, const pt<T> &a, const pt<T> &b)
    {
        if ((a - circ.c).toleft(b - circ.c) == 0)
            return 0.0l;
        const auto ina = circ.is_in(a), inb = circ.is_in(b);
        const line<T> ab = {a, b - a};
        if (ina && inb)
            return ((a - circ.c) % (b - circ.c)) / 2;
        if (ina && !inb)
        {
            const auto t = circ.inter(ab);
            const pt<T> p = t.size() == 1 ? t[0] : t[1];
            const ld ans = ((a - circ.c) % (p - circ.c)) / 2;
            const ld th = (p - circ.c).ang(b - circ.c);
            const ld d = circ.r * circ.r * th / 2;
            if ((a - circ.c).toleft(b - circ.c) == 1)
                return ans + d;
            return ans - d;
        }
        if (!ina && inb)
        {
            const pt p = circ.inter(ab)[0];
            const ld ans = ((p - circ.c) % (b - circ.c)) / 2;
            const ld th = (a - circ.c).ang(p - circ.c);
            const ld d = circ.r * circ.r * th / 2;
            if ((a - circ.c).toleft(b - circ.c) == 1)
                return ans + d;
            return ans - d;
        }
        const auto p = circ.inter(ab);
        if (p.size() == 2 && Segment<T>{a, b}.dis(circ.c) <= circ.r + eps)
        {
            const ld ans = ((p[0] - circ.c) % (p[1] - circ.c)) / 2;
            const ld th1 = (a - circ.c).ang(p[0] - circ.c), th2 = (b - circ.c).ang(p[1] - circ.c);
            const ld d1 = circ.r * circ.r * th1 / 2, d2 = circ.r * circ.r * th2 / 2;
            if ((a - circ.c).toleft(b - circ.c) == 1)
                return ans + d1 + d2;
            return ans - d1 - d2;
        }
        const ld th = (a - circ.c).ang(b - circ.c);
        if ((a - circ.c).toleft(b - circ.c) == 1)
            return circ.r * circ.r * th / 2;
        return -circ.r * circ.r * th / 2;
    };

    ld ans = 0;
    for (int i = 0; i < poly.p.size(); i++)
    {
        const pt<T> a = poly.p[i], b = poly.p[poly.nxt(i)];
        ans += cal(circ, a, b);
    }
```

```
    return ans;
}

// 多边形面积并
// 轮廓积分，复杂度 O(n^2logn)，n为边数
// ans[i] 表示被至少覆盖了 i+1 次的区域的面积
template <class T>
std::vector<ld> area(const std::vector<Polygon<T>> &polys)
{
    const int siz = polys.size();
    std::vector<std::vector<std::pair<pt<T>, pt<T>>>> segs(siz);
    const auto check = [](const pt<T> &u, const Segment<T> &e)
    { return !((u < e.a && u < e.b) || (u > e.a && u > e.b)); };

    auto cut_edge = [&](const Segment<T> &e, const int i)
    {
        const line<T> le{e.a, e.b - e.a};
        std::vector<std::pair<pt<T>, int>> evt;
        evt.push_back({e.a, 0});
        evt.push_back({e.b, 0});
        for (int j = 0; j < polys.size(); j++)
        {
            if (i == j)
                continue;
            const auto &pj = polys[j];
            for (int k = 0; k < pj.p.size(); k++)
            {
                const Segment<T> s = {pj.p[k], pj.p[pj.nxt(k)]};
                if (le.toleft(s.a) == 0 && le.toleft(s.b) == 0)
                {
                    evt.push_back({s.a, 0});
                    evt.push_back({s.b, 0});
                }
                else if (s.isCross1(le))
                {
                    const line<T> ls{s.a, s.b - s.a};
                    const pt<T> u = le.inter(ls);
                    if (le.toleft(s.a) < 0 && le.toleft(s.b) >= 0)
                        evt.push_back({u, -1});
                    else if (le.toleft(s.a) >= 0 && le.toleft(s.b) < 0)
                        evt.push_back({u, 1});
                }
            }
        }
        std::sort(evt.begin(), evt.end());
        if (e.a > e.b)
            std::reverse(evt.begin(), evt.end());
        int sum = 0;
        for (int i = 0; i < evt.size(); i++)
        {
            sum += evt[i].second;
            const pt<T> u = evt[i].first, v = evt[i + 1].first;
            if (!(u == v) && check(u, e) && check(v, e))
                segs[sum].push_back({u, v});
            if (v == e.b)
                break;
        }
    };

    for (int i = 0; i < polys.size(); i++)
    {
        const auto &pi = polys[i];
        for (int k = 0; k < pi.p.size(); k++)
        {
            const Segment<T> ei = {pi.p[k], pi.p[pi.nxt(k)]};
            cut_edge(ei, i);
        }
    }
    std::vector<ld> ans(siz);
    for (int i = 0; i < siz; i++)
    {
        ld sum = 0;
        std::sort(segs[i].begin(), segs[i].end());
        for (int j = 0; j < segs[i].size();)
        {
            int k = j;
            while (k < segs[i].size() && segs[i][k] == segs[i][j])
            {
                k++;
            }
```

```
            int count = k - j;
            sum += segs[i][j].first % segs[i][j].second;
            // 将多余的 count-1 个线段传递到下一层
            if (i + 1 < siz && count > 1)
            {
                for (int l = 0; l < count - 1; ++l)
                {
                    segs[i + 1].push_back(segs[i][j]);
                }
            }
            j = k;
        }
        ans[i] = sum / 2;
    }
    return ans;
}

// 圆面积并
// 轮廓积分，复杂度 O(n^2logn)
// ans[i] 表示被至少覆盖了 i+1 次的区域的面积
template <class T>
std::vector<ld> area(const std::vector<Circle<T>> &circs)
{
    const int siz = circs.size();
    using arc_t = std::tuple<pt<T>, ld, ld, ld>;
    std::vector<std::vector<arc_t>> arcs(siz);
    const auto eq = [](const arc_t &u, const arc_t &v)
    {
        const auto [u1, u2, u3, u4] = u;
        const auto [v1, v2, v3, v4] = v;
        return u1 == v1 && fabs(u2 - v2) <= eps && fabs(u3 - v3) <= eps && fabs(u4 - v4) <= eps;
    };

    auto cut_circ = [&](const Circle<T> &ci, const int i)
    {
        std::vector<std::pair<ld, int>> evt;
        evt.push_back({-pi, 0});
        evt.push_back({pi, 0});
        int init = 0;
        for (int j = 0; j < circs.size(); j++)
        {
            if (i == j)
                continue;
            const Circle<T> &cj = circs[j];
            if (ci.r < cj.r - eps && ci.relation(cj) >= 3)
                init++;
            const auto inters = ci.inter(cj);
            if (inters.size() == 1)
                evt.push_back({atan2l((inters[0] - ci.c).y, (inters[0] - ci.c).x), 0});
            if (inters.size() == 2)
            {
                const pt<T> dl = inters[0] - ci.c, dr = inters[1] - ci.c;
                long double argl = atan2l(dl.y, dl.x), argr = atan2l(dr.y, dr.x);
                if (fabs(argl + pi) <= eps)
                    argl = pi;
                if (fabs(argr + pi) <= eps)
                    argr = pi;
                if (argl > argr + eps)
                {
                    evt.push_back({argl, 1});
                    evt.push_back({pi, -1});
                    evt.push_back({-pi, 1});
                    evt.push_back({argr, -1});
                }
                else
                {
                    evt.push_back({argl, 1});
                    evt.push_back({argr, -1});
                }
            }
        }
        std::sort(evt.begin(), evt.end());
        int sum = init;
        for (int i = 0; i < evt.size(); i++)
        {
            sum += evt[i].second;
            if (fabs(evt[i].first - evt[i + 1].first) > eps)
                arcs[sum].push_back({ci.c, ci.r, evt[i].first, evt[i + 1].first});
            if (fabs(evt[i + 1].first - pi) <= eps)
                break;
```

```
        }
    };

    const auto oint = [](const arc_t &arc)
    {
        const auto [cc, cr, l, r] = arc;
        if (fabs(r - l - pi - pi) <= eps)
            return 2.0l * pi * cr * cr;
        return cr * cr * (r - l) + cc.x * cr * (sinl(r) - sinl(l)) - cc.y * cr * (cosl(r) - cosl(l));
    };

    for (int i = 0; i < circs.size(); i++)
    {
        const auto &ci = circs[i];
        cut_circ(ci, i);
    }
    std::vector<ld> ans(siz);
    for (int i = 0; i < siz; i++)
    {
        ld sum = 0;
        std::sort(arcs[i].begin(), arcs[i].end());
        for (int j = 0; j < arcs[i].size();)
        {
            int k = j;
            while (k < arcs[i].size() && eq(arcs[i][k], arcs[i][j]))
            {
                k++;
            }
            int count = k - j;
            sum += oint(arcs[i][j]);

            // 将多余的 count-1 个圆弧传递到下一层
            if (i + 1 < siz && count > 1)
            {
                for (int l = 0; l < count - 1; ++l)
                {
                    arcs[i + 1].push_back(arcs[i][j]);
                }
            }
            j = k;
        }
        ans[i] = sum / 2;
    }
    return ans;
}

// 半平面交
// 排序增量法，复杂度 O(nlogn)
// 输入与返回值都是用直线表示的半平面集合

// 有向线段的左边作为合法的半平面
template <class T>
std::vector<line<T>> halfinter(std::vector<line<T>> l, const ld lim = 1e18)
{
    const auto check = [](const line<T> &a, const line<T> &b, const line<T> &c)
    { return a.toleft(b.inter(c)) < 0; };
    // // 无精度误差的方法，但注意取值范围会扩大到三次方
    // const auto check = [](const line<T> &a, const line<T> &b, const line<T> &c)
    // {
    //     const pt<T> p = a.v * (b.v % c.v), q = b.p * (b.v % c.v) + b.v * (c.v % (b.p - c.p)) - a.p * (b.v % c.v);
    //     return p.toleft(q) < 0;
    // };
    l.push_back({{-lim, 0}, {0, -1}});
    l.push_back({{0, -lim}, {1, 0}});
    l.push_back({{lim, 0}, {0, 1}});
    l.push_back({{0, lim}, {-1, 0}});
    std::sort(l.begin(), l.end());
    std::deque<line<T>> q;
    for (int i = 0; i < l.size(); i++)
    {
        if (i > 0 && l[i - 1].v.toleft(l[i].v) == 0 && l[i - 1].v * l[i].v > eps)
            continue;
        while (q.size() > 1 && check(l[i], q.back(), q[q.size() - 2]))
            q.pop_back();
        while (q.size() > 1 && check(l[i], q[0], q[1]))
            q.pop_front();
        if (!q.empty() && q.back().v.toleft(l[i].v) <= 0)
            return std::vector<line<T>>();
        q.push_back(l[i]);
    }
```

```
    while (q.size() > 1 && check(q[0], q.back(), q[q.size() - 2]))
        q.pop_back();
    while (q.size() > 1 && check(q.back(), q[0], q[1]))
        q.pop_front();
    return std::vector<line<T>>(q.begin(), q.end());
}

// 点集形成的最小最大三角形(任意三点组成的三角形面积最大值和最小值)
// 极角序扫描线，复杂度 O(n^2logn)
// 最大三角形问题可以使用凸包与旋转卡壳做到 O(n^2)
// 凸包与旋转卡壳做到 O(n^2)做法是:
// 固定凸包顶点 A，枚举另一个顶点 B 沿凸包移动时，
// 使面积 |(B-A)*(C-A)| 最大的顶点 C 在凸包上具有单调性，
// 因此可以使用旋转卡壳维护 C，使其指针只向一个方向移动。
template <class T>
std::pair<ld, ld> minmax_triangle(const std::vector<pt<T>> &vec)
{
    if (vec.size() <= 2)
        return {0, 0};
    std::vector<std::pair<int, int>> evt;
    evt.reserve(vec.size() * vec.size());
    ld maxans = 0, minans = std::numeric_limits<ld>::max();
    for (int i = 0; i < vec.size(); i++)
    {
        for (int j = 0; j < vec.size(); j++)
        {
            if (i == j)
                continue;
            if (vec[i] == vec[j])
                minans = 0;
            else
                evt.push_back({i, j});
        }
    }
    std::sort(evt.begin(), evt.end(), [&](const std::pair<int, int> &u, const std::pair<int, int> &v)
              {
        const pt du=vec[u.second]-vec[u.first],dv=vec[v.second]-vec[v.first];
        return argcmp<T>()({du.y,-du.x},{dv.y,-dv.x}); });
    std::vector<int> vx(vec.size()), pos(vec.size());
    for (int i = 0; i < vec.size(); i++)
        vx[i] = i;
    std::sort(vx.begin(), vx.end(), [&](int x, int y)
              { return vec[x] < vec[y]; });
    for (int i = 0; i < vx.size(); i++)
        pos[vx[i]] = i;
    for (auto [u, v] : evt)
    {
        const int i = pos[u], j = pos[v];
        const int l = std::min(i, j), r = std::max(i, j);
        const pt vecu = vec[u], vecv = vec[v];
        if (l > 0)
            minans = std::min(minans, fabs((vec[vx[l - 1]] - vecu) % (vec[vx[l - 1]] - vecv)));
        if (r < vx.size() - 1)
            minans = std::min(minans, fabs((vec[vx[r + 1]] - vecu) % (vec[vx[r + 1]] - vecv)));
        maxans = std::max({maxans, fabs((vec[vx[0]] - vecu) % (vec[vx[0]] - vecv)), fabs((vec[vx.back()] - vecu) % (vec[vx.back()
    ] - vecv))});
        if (i < j)
            std::swap(vx[i], vx[j]), pos[u] = j, pos[v] = i;
    }
    return {minans, maxans};
}

// 判断多条线段是否两两之间没有交点?
// 只要有任意一个交点 返回true 否则返回false(即两两都没有交)
// 扫描线，复杂度 O(nlogn)

// 可能存在精度问题 下面有更稳定的迭代器版本
template <class T>
bool inter(const std::vector<Segment<T>> &segs)
{
    if (segs.empty())
        return false;
    using seq_t = std::tuple<ld, int, Segment<T>>;
    const auto seqcmp = [](const seq_t &u, const seq_t &v)
    {
        const auto [u0, u1, u2] = u;
        const auto [v0, v1, v2] = v;
        if (fabs(u0 - v0) <= eps)
            return std::make_pair(u1, u2) < std::make_pair(v1, v2);
        return u0 < v0 - eps;
```

```
    };
    std::vector<seq_t> seq;
    for (auto seg : segs)
    {
        if (seg.a.x > seg.b.x + eps)
            std::swap(seg.a, seg.b);
        seq.push_back({seg.a.x, 0, seg});
        seq.push_back({seg.b.x, 1, seg});
    }
    std::sort(seq.begin(), seq.end(), seqcmp);
    ld x_now;
    auto cmp = [&](const Segment<T> &u, const Segment<T> &v)
    {
        if (fabs(u.a.x - u.b.x) <= eps || fabs(v.a.x - v.b.x) <= eps)
            return u.a.y < v.a.y - eps;
        return ((x_now - u.a.x) * (u.b.y - u.a.y) + u.a.y * (u.b.x - u.a.x)) * (v.b.x - v.a.x) < ((x_now - v.a.x) * (v.b.y - v.a.
     y) + v.a.y * (v.b.x - v.a.x)) * (u.b.x - u.a.x) - eps;
    };
    std::multiset<Segment<T>, decltype(cmp)> s{cmp};
    for (const auto [x, o, seg] : seq)
    {
        x_now = x;
        const auto it = s.lower_bound(seg);
        if (o == 0)
        {
            if (it != s.end() && seg.isCross2(*it))
                return true;
            if (it != s.begin() && seg.isCross2(*std::prev(it)))
                return true;
            s.insert(seg);
        }
        else
        {
            if (std::next(it) != s.end() && it != s.begin() && (*std::prev(it)).isCross2(*std::next(it)))
                return true;
            s.erase(it);
        }
    }
    return false;
}
// // 判断多条线段是否两两之间存在交点（Lambda Map 最终版）
// template <class T>
// bool inter(const std::vector<Segment<T>> &segs)
// {
//     if (segs.empty())
//         return false;
//     using seq_t = std::tuple<ld, int, Segment<T>>;

//     // 1. 事件点创建和排序（不变）
//     std::vector<seq_t> seq;
//     for (auto seg : segs)
//     {
//         if (seg.a.x > seg.b.x + eps)
//             std::swap(seg.a, seg.b);
//         seq.push_back({seg.a.x, 0, seg});
//         seq.push_back({seg.b.x, 1, seg});
//     }
//     std::sort(seq.begin(), seq.end(), [](const seq_t &u, const seq_t &v)
//               {
//         if (fabs(std::get<0>(u) - std::get<0>(v)) <= eps)
//             return std::get<1>(u) < std::get<1>(v);
//         return std::get<0>(u) < std::get<0>(v) - eps; });

//     ld x_now;
//     // 2. multiset 的比较器（不变）
//     auto cmp = [&](const Segment<T> &u, const Segment<T> &v)
//     {
//         if (fabs(u.a.x - u.b.x) <= eps || fabs(v.a.x - v.b.x) <= eps)
//             return u.a.y < v.a.y - eps;
//         return ((x_now - u.a.x) * (u.b.y - u.a.y) + u.a.y * (u.b.x - u.a.x)) * (v.b.x - v.a.x) < ((x_now - v.a.x) * (v.b.y - v.
//      a.y) + v.a.y * (v.b.x - v.a.x)) * (u.b.x - u.a.x) - eps;
//     };
//     std::multiset<Segment<T>, decltype(cmp)> s{cmp};

//     auto C = [](const Segment<T> &l, const Segment<T> &r)
//     {
//         // 使用 std::tie 提供一个稳定的、唯一的排序规则
//         return std::tie(l.a.x, l.a.y, l.b.x, l.b.y) <
//                std::tie(r.a.x, r.a.y, r.b.x, r.b.y);
//     };
```

```
//     // 【改动 2】：声明 map 时，将 lambda 的类型作为模板参数，
//     //             并将 lambda 对象本身作为构造函数参数
//     using m_iter = typename std::multiset<Segment<T>, decltype(cmp)>::iterator;
//     std::map<Segment<T>, m_iter, decltype(C)> seg_iters(C);

//     // ^^^^^^^^^^^^^^^^ 【核心修改在这里】 ^^^^^^^^^^^^^^^^

//     for (const auto &[x, o, seg] : seq)
//     {
//         x_now = x;
//         if (o == 0) // 处理左端点
//         {
//             auto it = s.insert(seg);
//             seg_iters[seg] = it; // map 现在可以正常工作

//             if (it != s.begin() && seg.isCross2(*std::prev(it)))
//                 return true;
//             if (std::next(it) != s.end() && seg.isCross2(*std::next(it)))
//                 return true;
//         }
//         else // 处理右端点
//         {
//             auto it = seg_iters.find(seg)->second; // 精确查找

//             auto prev_it = (it == s.begin()) ? s.end() : std::prev(it);
//             auto next_it = std::next(it);

//             if (prev_it != s.end() && next_it != s.end() && (*prev_it).isCross2(*next_it))
//             {
//                 return true;
//             }
//             s.erase(it); // 精确删除
//             seg_iters.erase(seg);
//         }
//     }
//     return false;
// }
// 推荐的 inCircle 实现 大整数实现
// 返回值：
// > 0: d在圆外
// < 0: d在圆内
// = 0: d在圆上
__int128 det(__int128 a, __int128 b, __int128 c,
             __int128 d, __int128 e, __int128 f,
             __int128 g, __int128 h, __int128 i)
{
    return a * (e * i - f * h) - b * (d * i - f * g) + c * (d * h - e * g);
}
template <class T>
int inCircle(pt<T> a, pt<T> b, pt<T> c, pt<T> d)
{
    // 平移 d 到原点，减少数值大小
    a = a - d;
    b = b - d;
    c = c - d;
    auto val = det(
        a.x, a.y, dis2(a),
        b.x, b.y, dis2(b),
        c.x, c.y, dis2(c));

    // 计算 a,b,c 的方向
    ll orient = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
    if (orient < 0)
        val = -val; // 若顺时针则反转符号

    return (val > 0) - (val < 0); // 统一返回 -1/0/1
}
```

4.9.2 单峰函数与三分

常见单峰函数：
二次函数：其最值在对称轴取得
绝对值函数：
$f(x)=\sum_{i=1}^{n} abs(x-a_i)$ 是单峰函数，其最值在所有 a_i 的中位数取得
整数三分

```
int calc(int x)
{

}
int search(int l, int r)
{
    const int corner = 2;
    while (r - l > corner)
    { // 精度到达三分间隔时停止
        int m1 = l + (r - l) / 3;
        int m2 = r - (r - l) / 3;

        if (calc(m1) > calc(m2))
        {
            r = m2; // 左侧更优
        }
        else
        {
            l = m1; // 右侧更优
        }
    }
    int res = l;
    for (int i = l + 1; i <= r; ++i)
    {
        if (calc(i) > calc(res))
        {
            res = i;
        }
    }
    return res;
}
```

实数三分

```
long double calc(long double x)
{
}

long double search(long double l, long double r, long double epsilon = 1e-9)
{
    while (r - l > epsilon)
    {
        long double m1 = l + (r - l) / 3;
        long double m2 = r - (r - l) / 3;
        if (calc(m1) > calc(m2))
        {
            r = m2; // 左侧更优
        }
        else
        {
            l = m1; // 右侧更优
        }
    }
    return (l + r) / 2; // 返回最优点
}
```

三分套三分

使用场景往往是面向二元函数 $f(x,y)$, 满足在固定 x 时 $f(x,y)$ 随 y 变化是单峰的，那么我们对固定的 x，可三分出最优的 y。同时我们定义 $g(x)=min/max(f(x,y))$，如果 $g(x)$ 也是单峰的，即对于每个 x 在取到最优的 y 以后仍然保持单峰，那么就可以采用三分套三分。

结论：多个下凸函数的 max 仍然是下凸函数，多个上凸函数的 min 仍然是上凸函数

5 概率

5.1 马尔可夫性

对于一个 x=0 不断进行操作，a 的概率 +1,b 的概率终止，最后 x 的期望就是 a/b

设最后的期望是 E，那么 $E=\frac{a}{a+b}(1+E)+\frac{b}{a+b}\cdot 0$, 解方程即可得出 E=a/b

6 附件

6.1 组合数

part1 常用公式

1. $\binom{n}{m}=\frac{n^{\underline{m}}}{m!}=\frac{n!}{(n-m)!m!}$
2. $\binom{n}{m}=\binom{n-1}{m-1}+\binom{n-1}{m}$
3. $\binom{n}{m}=\frac{n}{m}\binom{n-1}{m-1}$
4. $\binom{n}{m}=\frac{n-m+1}{m}\binom{n}{m-1}$
5. $\binom{n}{0}+\binom{n}{1}+\ldots+\binom{n}{n}=2^n$
6. $\binom{n}{0}+\binom{n}{2}+\binom{n}{4}+\ldots=\binom{n}{1}+\binom{n}{3}+\binom{n}{5}+\ldots=2^{n-1}$
7. $$(a+b)^n=\sum_{k=0}^{n}\binom{n}{k}a^{n-k}b^k(\quad)$$
8. $$\sum_{i=m}^{n}\binom{i}{a}=\binom{n+1}{a+1}-\binom{m}{a+1}(\quad)$$

part 2 小球放盒问题

9. 有 n 个相同的球,k 个不同的盒子，把 n 个球放到盒子里，盒子不允许为空，有多少种方案。这个问题等价于求 $n=a_1+a_2+\ldots a_k, a_i$ 不能为 0 的方案数。隔板法，插空法。把 n 个相同的球放在一行，那么这 n 个球中间有 n-1 个空，那么在这 n-1 个空中选 k-1 个空，放 k-1 个隔板，就把这 n 个球分成了 k 份。所以方案数为 C(n-1,k-1) , 组合数公式，表示从 n-1 个取 k-1 个的方案数。例题: 牛客网 https://ac.nowcoder.com/acm/contest/553/D
10. 有 n 个相同的球，k 个不同的盒子，把 n 个球放到盒子里，盒子允许为空，有多少种方案。和问题 1 方法一样，这个允许盒子为空。n=a1+a2+a3+…ak ,ai 可以为 0 的方案数。这个题等价于 有 n+k 个相同的球，k 个不同盒子，盒子不许为空的方案数。因为：把这 n+k 个球分到 k 个盒子之后，把每个盒子里的球的数量都减一，那么球的总数就是 n 个了，盒子里的球就可能是空的了。所以方案数为 C(n+k-1,k-1) . 例题: 牛客网 https://ac.nowcoder.com/acm/contest/553/D
11. 有 n 个相同的球，k 个相同的盒子，把 n 个球放到盒子里，盒子允许为空，有多少种方案。

    这是一道动态规划题，用 dp[i][j] 表示把 n 个球放到不超过 k 个盒子里的方案数。我们可以根据有没有空盒子列出下面这个转移方程: $dp[i][j]=dp[i][j-1]+dp[i-j][j]$, 解释一下：$dp[i][j-1]$ 表示有空盒子，那么就是 i 个球放到不超过 j-1 个盒子的方案数 dp[i-j][j] 表示

没有空盒子，那么每个盒子最少要有 1 个球，那么先把每个盒子放一个球，还剩下 i-j 个球，把剩下的 i-j 个球分到不超过 j 个盒子。

4. 有 n 个相同的球，k 个相同的盒子，把 n 个球放到盒子里，盒子不允许为空，有多少种方案。和第 3 个方法一样，也是动态规划，dp[i][j] 表示的意义和第 3 题一样。答案应该是 dp[n][k]-dp[n][k-1]. 显然，（n 个球放到 k 个相同的盒子，盒子允许为空的方案数）减去（n 个球放到 k-1 个盒子，盒子允许为空的方案数）就是（n 个球放到 k 个盒子，盒子不允许为空的方案数）。

5. 有 n 个不同的球，k 个相同的盒子，把 n 个球放到盒子里，盒子不允许为空，有多少种方案。这个题是第二类 Stirling 数，用 S（n,k）表示 n 个不同的球，放到 k 个不同的盒子，盒子不允许为空的方案数。那么 S(n,k)=S（n-1,k-1）+kS(n-1,k) 1<=k<=n S(0,0)=1,S(i,0)=0 解释一下怎么理解这个状态转移方程，第 n 个球可以有两种状态，第 n 个球单独在一个盒子里，第 n 个球所在盒子至少有两个。S(n-1,k-1) 就是第 n 个球单独在一个盒子里，那么就剩 n-1 个不同球，分到剩下的 k-1 个盒子里。kS(n-1,k)： 先把前 i-1 个分配到 k 个盒子里，方案数是 S(n-1,k) ，第 n 个球然后就有 k 种方法，总方案数就是 k*S(n-1,k) dp 打出一个二维数组的表，就能算出答案

6. 有 n 个不同的球，k 个相同的盒子，把 n 个球放到盒子里，盒子允许为空，有多少种方案。和第 5 题方法一样，答案就是 S(n,1)+S(n,2)+⋯..S(n,k)

7. 有 n 个不同的球，k 个不同的盒子，把 n 个球放到盒子里，盒子不允许为空，有多少种方案。和第 5 题的区别是盒子不同，那么答案就是 S(n,k)*k!

8. 有 n 个不同的球，k 个不同的盒子，把 n 个球放到盒子里，盒子允许为空，有多少种方案。k 的 n 次方，每个球都有 k 中选择。

6.2 因数个数 估算

统计 n 的所有因数

solution1 sqrt 匹配

solution2 利用 $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$ 先分解质因数，然后凑答案，复杂度 O(S) ,S 为因子个数

下表非具体数值，仅用于复杂度估算

n	max(S)
1e6	3e2
1e9	2e3
1e12	9e3
1e18	2e5